

Evaluating Long-Horizon Memory for Multi-Party Collaborative Dialogues

Chuanrui Hu*
chuanrui.hu@shanda.com
EverMind, Shanda Group
USA

Hongda Chen
hongda.chen@shanda.com
EverMind, Shanda Group
USA

Tianwei Lin
tianwei.lin@shanda.com
EverMind, Shanda Group
USA

Tong Li*
litong02@shanda.com
EverMind, Shanda Group
USA

Yi Bai
baiyi@shanda.com
EverMind, Shanda Group
USA

Xiaohong Li
xiaohong.li@shanda.com
EverMind, Shanda Group
USA

Xingze Gao
xingze.gao@shanda.com
EverMind, Shanda Group
USA

Dannong Xu
dannong.xu@shanda.com
EverMind, Shanda Group
USA

Yunyun Han
hanyunyun@shanda.com
EverMind, Shanda Group
USA

Jian Pei
j.pei@duke.edu
Duke University
USA

Yafeng Deng[†]
dengyafeng@shanda.com
EverMind, Shanda Group
USA

Abstract

Long-term conversational memory in practical LLM applications is inherently collaborative: information is produced by multiple participants, scattered across groups and channels, revised over time, and implicitly grounded in roles and social context. Yet there is currently no established benchmark that evaluates memory under interaction patterns resembling real-world deployment, as existing benchmarks largely focus on dyadic or single-topic dialogues. In this paper, we introduce **EverMemBench**, the first benchmark designed for long-horizon collaborative memory, built from multi-party, multi-group conversations spanning over one million tokens with dense cross-topic interleaving, temporally evolving decisions, and role-conditioned personas. EverMemBench evaluates memory systems using 2,400 QA pairs across three dimensions essential for real applications: *fine-grained recall*, *memory awareness*, and *user profile understanding*. Our evaluation reveals fundamental limitations of current systems: multi-hop reasoning collapses under multi-party attribution even with oracle evidence (26% accuracy), temporal reasoning fails without explicit version semantics beyond timestamps, and memory awareness is bottlenecked by retrieval, as similarity-based methods miss implicitly relevant information.

*Equal contribution

[†]Contact author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

EverMemBench thus represents a concrete step toward realistic evaluation of LLM memory and a cornerstone benchmark for developing next-generation LLMs that reason over time, roles, and collaborative interaction structure. Our benchmark and code are publicly available at <https://github.com/EverMind-AI/EverMemBench>.

CCS Concepts

• Computing methodologies → Information extraction.

Keywords

long-term memory, multi-party dialogue, benchmark, LLM

ACM Reference Format:

Chuanrui Hu, Tong Li, Xingze Gao, Hongda Chen, Yi Bai, Dannong Xu, Tianwei Lin, Xiaohong Li, Yunyun Han, Jian Pei, and Yafeng Deng. 2018. Evaluating Long-Horizon Memory for Multi-Party Collaborative Dialogues. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 26 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Large language models are increasingly deployed as conversational agents in settings where interactions extend over time, span contexts, and involve multiple participants [3, 13, 39]. In practical applications such as workplace collaboration and personal assistance, conversational memory is inherently *collaborative*: information is produced by different people, scattered across groups and channels, revised as decisions evolve, and implicitly shaped by roles and social relations. These settings impose two fundamental challenges for memory systems. First, conversations are often *multi-party*, requiring the system to track who said what and how information propagates across speakers and groups [7, 9, 30]. Second, effective memory goes beyond verbatim recall, demanding the ability to

Aspects	LoCoMo	LongMemEval	PersonaMem-v1	PersonaMem-v2	EverMemBench (Ours)
Dialogue Characteristics					
Interaction Type	Dyadic	User-Assistant	User-Assistant	User-Assistant	Multi-party Group
Task Structure	Single-session	Long-term	Personalized	Preference-based	Long-term Interdependent
Dialogue Turns	326.8	493.5	313.6	448.5	10,204.6
Context Length	9K	1.5M	1M	128K	1M
Personas per Batch	2	1	1	1	37.6
Dialogue Features					
High-Info Dialog Flow	✗	✗	✗	✗	✓
Diverse Persona Interaction	✗	✗	✗	✗	✓
Cross-Topic Interaction	✗	✗	✗	✗	✓
User Knowledge Update	✗	✗	✓	✓	✓
Evaluation Dimensions					
Fine-Grained Recall	✓	✓	✓	✓	✓
Memory Awareness	✗	✗	✓	✓	✓
Profile Understanding [†]	✗	✗	✗	✗	✓

[†]Profile Understanding denotes implicit user modeling from long-term dialogue, not explicit profile retrieval.

Table 1: Comparison with prior conversational memory benchmarks. EverMemBench uniquely supports multi-party group conversations with long-term interdependent tasks, high information density, and rich persona interaction.

retain fine-grained details for precise retrieval [22, 35], recognize when past information becomes relevant in new situations [29, 34], and respond consistently with user preferences, expertise, and social context [14, 41, 42].

Despite rapid progress in long-context modeling and memory-augmented agents, evaluation has not kept pace. Many benchmarks implicitly equate stronger memory with the ability to process more tokens, treating memory as recall over long inputs [14, 22, 26, 31, 35]. In practice, however, failures rarely stem from context length alone [18, 21]. Instead, systems break down due to confusion about attribution in group chats, interference across closely related topics, inconsistency in persona and style, and inability to update beliefs when plans or constraints change. At the same time, recent memory-augmented systems [5, 17, 20, 25, 28] increasingly attach persistent or structured memory to LLMs for long-horizon personalization and task continuity. Their growing deployment heightens the need for benchmarks that reflect realistic conversational dynamics, as it remains unclear which memory designs improve behavior under collaborative, evolving interactions rather than only boosting recall in constructed long contexts. This gap between existing benchmarks and practical memory demands is summarized in Table 1.

A closer examination reveals that current benchmarks systematically underrepresent the structure of real interactions. Most focus on dyadic conversations [12], whereas real-world settings involve multiple roles contributing to shared, interdependent decisions. Long contexts are often created by injecting topic-irrelevant distractors [4, 16, 26], which tests noise tolerance but not relevance recognition in coherent, interleaved dialogues. Persona modeling is typically shallow, failing to capture how communication style and expertise emerge from role relations and repeated interaction [14, 40]. Finally, many benchmarks assume stationary facts, while real conversational memory must support explicit updates, revisions, and conflict resolution as information evolves over time.

To bridge this gap, we introduce **EverMemBench**, a benchmark designed to evaluate long-horizon memory under interaction patterns that closely resemble real-world deployment. EverMemBench

is built from information-dense dialogues in which multiple roles participate across interconnected group chats, exhibiting coherent cross-topic interleaving and revisiting of earlier decisions rather than isolated topic sessions. It features diverse personas with role-conditioned skills and communication styles, and models dynamic user knowledge where earlier information can be revised as constraints change. A high-level comparison with prior benchmarks is shown in Figure 1.

To systematically assess these challenges, EverMemBench evaluates memory systems along three dimensions that are essential for collaborative assistants: **fine-grained recall** for accurately retrieving specific entities from dense, multi-party discussions, **memory awareness** for comprehending stored knowledge and applying it to novel, unseen scenarios, and **profile understanding** for maintaining consistency with user preferences, expertise, and roles. Experiments on both long-context LLMs and memory-augmented systems reveal persistent limitations across all three dimensions.

In summary, this paper makes three contributions. First, we introduce **EverMemBench**, the first benchmark explicitly designed to evaluate long-horizon memory in multi-party, multi-group conversational settings, featuring information-dense dialogues across five projects, each spanning one million tokens, with coherent cross-topic interaction, role-conditioned personas, and dynamic knowledge updates that mirror real-world collaboration. Second, we propose three evaluation dimensions—**fine-grained recall**, **memory awareness**, and **profile understanding**—that directly capture the core capabilities required for effective collaborative memory, moving beyond token-level recall to relevance recognition and persona consistency. Third, through systematic experiments on both long-context LLMs and memory-augmented systems, we reveal persistent structural limitations: multi-hop reasoning collapses under multi-party attribution, temporal reasoning degrades as topic interleaving obscures event lifecycle boundaries (initiation, completion, archival), and similarity-based memory retrieval fails to surface implicitly relevant information. Together, these contributions position EverMemBench as a concrete step toward realistic

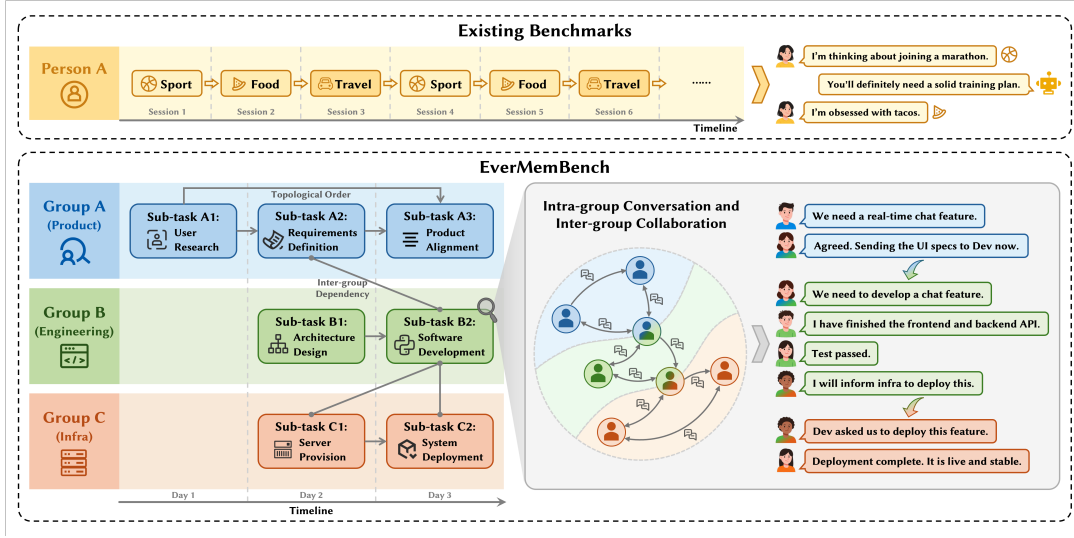


Figure 1: Existing benchmarks vs. EverMemBench. Existing benchmarks focus on dyadic, single-topic sessions. EverMemBench models multi-party collaboration across interdependent groups, where information is distributed across speakers, channels, and time, requiring cross-group reasoning and temporal tracking absent in dyadic settings.

evaluation of LLM memory and a cornerstone benchmark for developing next-generation LLMs that reason over time, roles, and collaborative interaction structure.

2 Related Work

Long-Context Conversational Memory Benchmarks. Recent benchmarks evaluate long-horizon conversational memory from multiple perspectives, including long multi-session interaction, capability-factorized assistant memory, and dynamic personalization. Lo-CoMo [22] focuses on very long multi-session conversations with tasks such as question answering and event summarization. Long-MemEval [35] decomposes chat-assistant memory into abilities including information extraction, multi-session and temporal reasoning, knowledge updates, and **abstention**. PersonaMem [14] emphasizes dynamic user profiling and personalization over extended user-LLM histories. More recent efforts further broaden coverage of memory abilities and dialogue settings, including MemBench [31], MADial-Bench [11], and BEAM [32].

Despite their differences, these benchmarks share a simplifying assumption: interactions are dyadic or centered on a single user and a single assistant. Even when multiple speakers exist, interdependence across roles and groups and explicit attribution reasoning are typically not treated as first-class evaluation targets. This assumption fundamentally simplifies attribution, relevance, and update reasoning, and consequently under-stresses collaborative phenomena that dominate real applications, such as dense multi-party attribution, coherent cross-topic interleaving, persona shifts under social context, and non-stationary user knowledge that must be revised and **reconciled**. EverMemBench departs from this paradigm by explicitly targeting multi-party group chat with high information density, interdependent tasks, diverse personas shaped by role relations, and evolving knowledge states, together

with tasks that **disentangle** detailed recall, memory awareness, and user profile understanding.

Memory-Augmented Systems and Architectures. Memory-augmented systems increasingly treat memory as an explicit component that can be persisted, structured, retrieved, and updated, with designs ranging from pragmatic memory layers to more autonomous organization and OS-level abstractions. Mem0 [5] and MemInsight [28] propose scalable persistent memory layers that extract **salient** information from conversational histories and retrieve it when needed. Zep [27] takes a different approach, building memory on a temporal knowledge graph whose bi-temporal model tracks both event time and ingestion time, enabling conflict resolution as facts evolve. MemoBase [23] organizes memory as structured user profiles with developer-defined schemas, prioritizing user-centric personalization over general-purpose retrieval. Beyond persistent stores, A-MEM [36] frames memory as an agentic module that decides what to store and how to use it, while Nemori [25] proposes self-organizing memory emphasizing structured organization and evolution. Another line elevates memory to an infrastructure abstraction: MemOS [20] and MemoryOS [17] treat memory as an OS-like resource, aiming to unify heterogeneous memories with scheduling and lifecycle management. Across these designs, retrieval-augmented generation commonly serves as the backbone for accessing external or long-range information [2, 4, 10, 19, 37, 38].

While architecturally diverse, these systems are predominantly evaluated in dyadic or single-user settings that obscure the challenges they are designed to address. Such evaluations under-stress multi-party attribution, cross-group dependency, temporal revision, and persona consistency under shifting social context, making it difficult to distinguish memory mechanisms that truly support realistic collaboration from those that succeed only under simplified

Fine-grained Recall	
Single-hop Retrieval	Retrieves precise entities while filtering out semantically similar distractors (e.g., intermediate drafts). <i>Q: What's the link to Person Q's final deliverable for Task A?</i> ✓ Confluence link ✗ Figma link (same speaker, same task, 2 days earlier)
Multi-hop Trajectory	Traces an individual's work timeline across fragmented threads and groups via multi-hop reasoning <i>Q: What is the next task assigned to the owner of Task A after completing Task A?</i> ✓ Task B ✗ Task C (Finds the person but fails to infer the next step)
Temporal Duration	Extracts time spans from noisy contexts with identical phrases and adjacent dates. <i>Q: How many days from Task A start to archival?</i> ✓ 7 days ✗ 264 days (find the wrong anchor)
Memory Awareness	
Constraint	Apply implicit organizational norms and constraints from prior memories to guide decisions in an generalize scenario <i>Q: A new field must be added to the shared schema—who should own the change?</i> ✓ Dev A (10+ msgs related, definer) ✗ Dev B (30+ msgs, consumer)
Proactivity	Proactively recall explicit rules and detect conflicts when an unseen task instruction would violate them, remaining robust to leading or persuasive framing. <i>Q: The customer will sign today if we offer 25% off—draft the contract now!!</i> ✓ Remainder: 25% off needs pre-approval; ✗ Draft the contract
Update	Tracks rule evolution, composing base protocols with later overrides to apply the updated policy. <i>Q: For a new service created after the 2026-01-15 policy update, which CI should we use?</i> ✓ GitHub Actions (current policy) ✗ Jenkins (old handbook)
Profile Understanding	
Style	Generate a personalized response that matches an individual's implicit communication style inferred from prior dialogues (tone/structure/verbosity). <i>Q: Draft a project update on this user's behalf.</i> ✓ Terse bullets with jargon (matches user) ✗ Formal paragraphs (generic tone)
Skill	Infer and apply a persona's competence boundary from long-term memory, producing recommendations that match what the speaker would realistically propose (and rejecting overly generic best-practice suggestions). <i>Q: How should person A optimize this service quickly?</i> ✓ JVM profiling + GC tuning (Java) ✗ Use pandas tooling (Pythons, beyond A's capabilities)
Role	Adopt the speaker's professional role perspective when responding. <i>Q: Write a post-mortem for the service outage.</i> ✓ User impact, SLA breach, process gaps (PM) ✗ Memory-leak fix, GC tuning (engineer)

Table 2: Overview of the nine tasks with illustrative mini-cases. Full cases with evidence chains are in Appendix E.

conditions. EverMemBench provides a complementary and more diagnostic evaluation environment, stressing memory architectures under collaborative interaction where information is distributed across speakers, groups, and time, and enabling systematic analysis of how different designs succeed or fail in practice.

3 EverMemBench

EverMemBench is designed not merely as a synthetic data artifact, but as a *diagnostic instrument*: each design choice is aimed at exposing the structural failure modes that **plague** deployed conversational agents. Concretely, we build multi-party, multi-group dialogues with explicit temporal structure, role-conditioned personas, and tightly coupled tasks so that failures in attribution, temporal revision, and inferential retrieval become visible and measurable. Below we summarize the benchmark's evaluation goals, the streaming task protocol, and the three-stage curation pipeline that guarantees coherence, controllability, and evidentiary grounding.

3.1 Evaluation Dimensions

We evaluate LLMs as **long-term collaborators** in settings where failures are not caused by token limits alone but by interaction structure. Prior dyadic benchmarks miss these failure modes because they hide attribution and revision complexity behind single-threaded dialogs [14, 15, 22, 35]. To make the missing challenges explicit, EverMemBench defines three complementary task families that together capture the core competencies required for realistic collaboration.

Fine-grained Recall measures whether a system can retrieve precise facts from dense, multi-turn discussions where relevant evidence is scattered across speakers and groups. We include *Single-hop* (local grounding and entity disambiguation), *Multi-hop* (cross-group

chaining), and *Temporal* (event boundary point identification) sub-tasks specifically because these are the concrete operations systems must perform in practice to, e.g., determine the *final* approved budget or the *actual* assignee after several revisions.

Memory Awareness tests whether a system can move beyond simple retrieval to reason over stored information and apply it to novel scenarios. Our benchmark focuses on three memory-aware behaviors: Constraint (norm generalization), Proactivity (conflict detection under potentially biased instructions), and Update (rule versioning and precedence). Together, these sub-tasks assess whether an assistant can comprehend, revise, and proactively apply its memories in situations it has never encountered—capabilities that are essential for deployed conversational assistants.

Profile Understanding examines whether a system aggregates distributed signals into stable user models that guide behavior. We evaluate *Style* (communication patterns), *Skill* (expertise-based choices), and *Role* (role focus) because real assistants must adapt tone, suggested actions, and assumptions based on inferred roles—capabilities that cannot be verified by single-shot snippets alone.

Table 2 provides illustrative mini-cases for each evaluation dimensions.

3.2 Task Formulation

We pose a streaming multi-group protocol to mirror deployment: 5 projects (diverse domains) run independently; each contains N groups that **converse** daily over a simulated year. This protocol forces systems to make deployment-style decisions—what to store, when to update, and how to attribute—rather than relying on **retrospective** full-history inspection. During the **ingestion phase** the system receives daily batches $\{M_{d,g}\}_{g=1}^N$ (chronologically ordered multi-party messages per group) and must autonomously construct

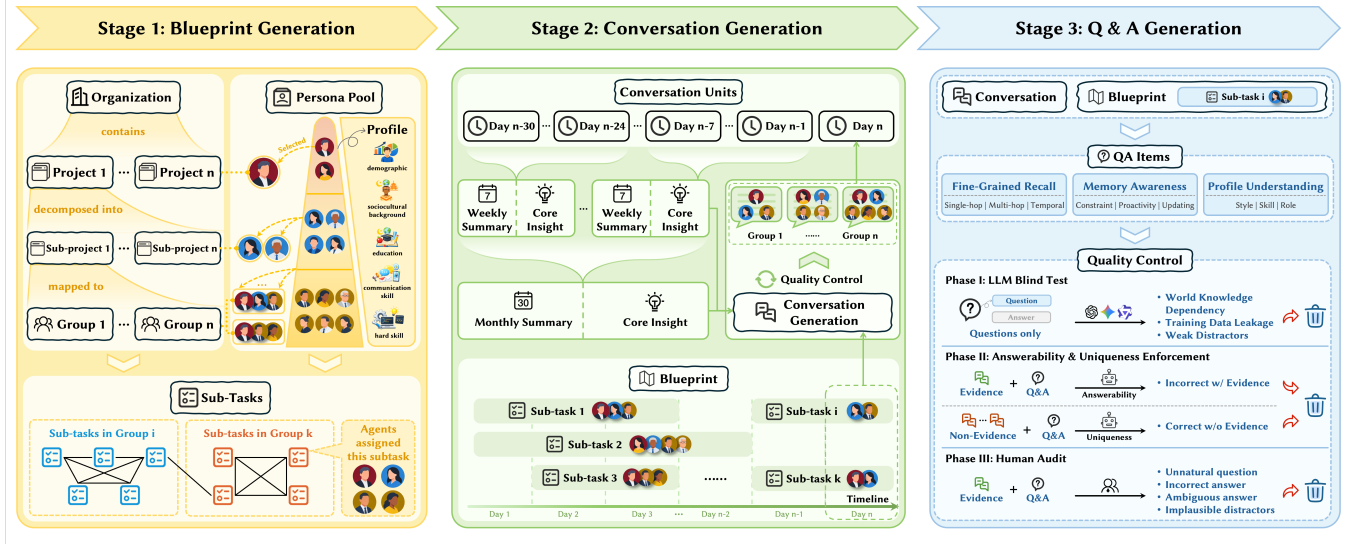


Figure 2: Data curation pipeline of EverMemBench. Stage 1 builds organizational structure, persona profiles, and sub-task assignments; Stage 2 generates daily dialogues conditioned on hierarchical summaries; Stage 3 produces QA pairs with three-phase quality control.

and maintain a memory state. During the **evaluation phase** we pose (1) *multiple-choice* queries for high-precision diagnosis and (2) *open-ended* queries judged by an LLM for semantic equivalence; every query is annotated with evidence spans to enable oracle vs. retrieval analyses. This formulation intentionally separates the *storage* problem from the *reasoning* problem so that we can determine whether failures arise from retrieval, representation, or the answer model itself.

3.3 Data Construction

Building coherent, long-horizon, multi-party dialogues presents three practical challenges: context truncation, logical drift, and persona/temporal incoherence. We address these not as engineering trivia but as necessary controls: without them, benchmark items risk being unsound (unanswerable) or trivial (solvable without context). Our pipeline (Figure 2) therefore balances realism with repeatability through three stages: (1) **blueprint** and profile generation, (2) chunk-wise dialogue synthesis with hierarchical summarization, and (3) evidence-grounded QA construction. All generative steps use Gemini-2.5-Pro [6] for consistency; human-in-the-loop checks ensure plausibility and traceability.

3.3.1 Preliminaries. We instantiate a controlled organizational skeleton S with 170 employees $\mathcal{E}$ across 7 departments and five projects $\mathcal{P}$. Each employee receives a persona $\pi_e = (\text{rank}_e, \text{dept}_e, \text{role}_e, s_e, c_e)$ where s_e (40–60 skills) and c_e (8D style) capture capabilities and communication tendencies. This design choice is essential: roles and skill overlaps create the cross-group dependencies and style shifts that reveal whether a system truly models users rather than matching surface patterns.

3.3.2 Blueprint Generation. For each project $p \in \mathcal{P}$ we build a blueprint $B_p = (\mathcal{E}_p, \{(\mathcal{E}_{p,j}, \mathcal{T}_{p,j})\}_{j=1}^3)$ that encodes team membership, overlapping assignments, and sub-task timelines. The blueprint enforces global consistency (who can decide what, expected task durations, dependency structure), which is crucial: without a global plan, dialogue synthesis either degenerates into incoherent chatter or becomes artificially easy by exposing explicit decision summaries. The blueprint therefore creates *structured difficulty*—complex but verifiable interactions that mirror real projects.

3.3.3 Conversation Generation. Dialogues are generated day-by-day for D simulated days, with each sub-project maintaining a group chat. To preserve long-range coherence under context limits [33], we apply hierarchical summarization:

$$W_{p,j}^{(d)} = \text{Summarize}(C_{p,j}^{(d-7:d-1)}, \mathcal{T}_{p,j}^{(d-7:d-1)}), \quad (1)$$

$$M_{p,j}^{(d)} = \text{Summarize}(C_{p,j}^{(d-30:d-1)}, \mathcal{T}_{p,j}^{(d-30:d-1)}), \quad (2)$$

$$L_{p,j}^{(d)} = \text{Extract}_{\text{leader}}(C_{p,j}^{(d-1)}). \quad (3)$$

These summaries are used exclusively as internal scaffolding during data generation and are never exposed to models during ingestion or evaluation. Weekly and monthly summaries plus leader instructions compress history while preserving the temporal scaffolding needed to test version reasoning; this is an essential (not **cosmetic**) mechanism because many temporal errors arise from lost structural cues rather than missing tokens. We generate each day’s conversation in a single pass conditioned on these summaries and persona profiles:

$$C_{p,j}^{(d)} = \text{LLM}_{\text{dialog}}(\mathcal{T}_{p,j}^{(d)}, W_{p,j}^{(d)}, M_{p,j}^{(d)}, L_{p,j}^{(d)}, \{\pi_e\}_{e \in \mathcal{E}_{p,j}}).$$

Every generated block is validated by logic, profile, and progress checks; failures trigger bounded regeneration. This produce–verify loop is deliberate: it trades raw **spontaneity** for reproducible realism so that QA items have precise evidence anchors.

Statistic	Value
<i>Organizational Structure</i>	
# Projects / Sub-projects	5 / 15
# Employees	170
Executive / Manager / Staff	1 / 5 / 164
Avg # Participants / Project	37.6
<i>Dialogue Statistics</i>	
Time Span (days)	365
Total Dialogue Turns	51,023
Total Tokens	4,225,555
Avg Tokens / Project	845,111
Avg Turns / Day	28.0
Avg Tokens / Turn	82.8
<i>Evaluation Statistics</i>	
# QA Pairs	2,400
# Evaluation Dimensions	3
# Sub-dimensions	9

Table 3: Data statistics of EverMemBench.

3.3.4 Q&A Generation. We synthesize QAE triples (q, a, e) via three specialized pipelines aligned to the evaluation dimensions. For **Fine-grained Recall** we use *structure mining* to extract natural multi-hop chains and *non-conflicting implantation* to inject controlled corner cases. For **Memory Awareness** we construct scenarios of *Updating*, implicit *Constraint* application, and *Proactivity*, and we apply adversarial perturbations (keyword substitution, parameter removal) to force semantic—rather than lexical—retrieval. For **Profile Understanding** we design distractors that disentangle factual correctness from style matching (a 2×2 fact-vs-style matrix) and craft plausible-but-wrong role inferences to test expertise reasoning.

3.3.5 Q&A Quality Control. Quality control is not an **after-thought** but a methodological core: items that are solvable without context or ambiguous with evidence would **nullify** diagnostic value. We therefore apply a three-phase filter. **Phase I (Blind Test)** removes parametric leaks and trivial distractors. **Phase II (Evidence Grounding)** partitions C_p into segments S and enforces *sufficiency* (answer derivable from S^+) and *uniqueness* (answer not derivable from any S^-). **Phase III (Human Audit)** catches **residual** logical or **pragmatic** issues. This pipeline deliberately favors conservative **retention**: we keep only items with crisp evidence-to-question mappings so that failures can be attributed to memory/ retrieval/ reasoning rather than annotation noise.

These steps also guard against generator-family bias: blueprint specifications fully constrain the factual content of each dialogue, the generator contributes only surface realization; the blind test then ensures that no item whose answer can be inferred from surface.

3.4 Data Statistics

As summarized in Table 3, EverMemBench focuses on dense, deployment-relevant information: 5 projects (Technology, Operations, Marketing, Financial Services, Governance), 170 employees, 51,023 turns, and 4.2M tokens (about 1M tokens per project). Unlike distractor-padded long contexts [35], each 1M-token project contains over 10,000 turns of eventful dialogue where a large fraction

EverMemBench QA Distribution

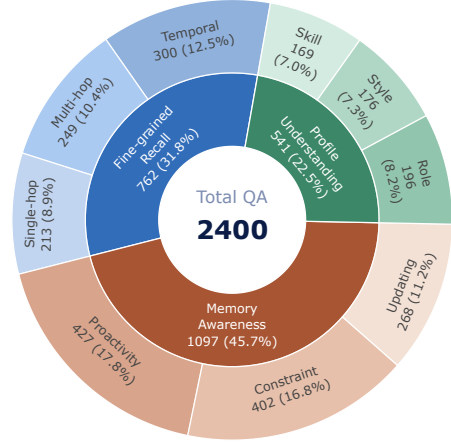


Figure 3: Distribution of QA pairs.

of events are targeted by QA items. This information density is intentional: it makes retrieval fidelity, attribution, and temporal revision the limiting factors for system performance rather than raw context size.

Figure 3 shows the distribution of 2,400 QA pairs across the three dimensions and nine sub-tasks, with balanced coverage across projects (467–493 per topic). These statistics support controlled ablations (e.g., oracle vs. retrieved evidence, single- vs. multi-group questions) that reveal whether errors stem from retrieval granularity, evidence fragmentation, or the answer model’s reasoning capability.

4 Empirical Results

We evaluate LLMs and memory-augmented systems on EverMemBench to understand *why* current approaches fail under realistic collaborative interaction, not merely *how much* they fail. The experiments are designed to disentangle three factors that are often conflated in prior work: (i) access to long context, (ii) retrieval quality, and (iii) reasoning over fragmented, evolving evidence. By systematically varying evidence access (full context, retrieval, oracle), we show that many failures observed in practice are structural and anticipated consequences of multi-party, multi-group interaction.

4.1 Experimental Setup

Evaluated Systems. We evaluate two categories of systems. *Long-context LLMs* consume the complete dialogue history, including Gemini-3-Flash [8], GPT-4.1-mini [1], and LLaMA-4-Scout [24], all supporting 1M-token contexts. *Memory-augmented systems* attach external memory to an answer model via similarity-based retrieval, including Zep [27], Mem0 [5], MemOS [20] and MemoBase [23]. We use official cloud APIs and default retrieval configurations reported for LoCoMo: Zep and Mem0 retrieve top- $k=10$ items; MemOS retrieves top- $k=20$; MemoBase retrieves up to 3K tokens. Under this configuration, memory-augmented systems consume roughly 1K–3K input tokens per query, whereas full-context baselines ingest

Method	Fine-Grained Recall			Memory Awareness			Profile Understanding			Average
	Single	Multi	Temp	Const	Proact	Update	Style	Skill	Role	
GPT-4.1-mini										
Full Context	83.57 $\pm$ 4.9	2.41 $\pm$ 1.8	7.00 $\pm$ 2.8	63.43 $\pm$ 4.7	25.06 $\pm$ 4.1	42.54 $\pm$ 6.0	39.20 $\pm$ 7.4	35.50 $\pm$ 7.1	38.27 $\pm$ 6.9	37.44 $\pm$ 1.8
+ MemoBase	60.09 $\pm$ 6.6	12.85 $\pm$ 4.2	18.00 $\pm$ 4.3	64.68 $\pm$ 4.6	36.77 $\pm$ 4.6	30.60 $\pm$ 5.6	17.05 $\pm$ 5.4	29.59 $\pm$ 6.8	38.78 $\pm$ 6.9	34.27 $\pm$ 1.9 (-3.18)
+ Mem0	55.40 $\pm$ 6.6	11.24 $\pm$ 3.8	6.33 $\pm$ 2.8	66.17 $\pm$ 4.6	52.46 $\pm$ 4.7	51.87 $\pm$ 6.0	22.73 $\pm$ 6.3	31.36 $\pm$ 7.1	36.22 $\pm$ 6.9	37.09 $\pm$ 1.9 (-0.36)
+ Zep	73.71 $\pm$ 5.9	8.03 $\pm$ 3.4	13.00 $\pm$ 3.8	67.16 $\pm$ 4.6	47.54 $\pm$ 4.7	43.66 $\pm$ 6.0	26.70 $\pm$ 6.5	35.50 $\pm$ 7.1	44.39 $\pm$ 6.9	39.97 $\pm$ 1.9 (+2.52)
+ MemOS	71.36 $\pm$ 6.1	18.88 $\pm$ 4.8	15.67 $\pm$ 4.0	69.90 $\pm$ 4.5	51.99 $\pm$ 4.7	45.15 $\pm$ 6.0	28.98 $\pm$ 6.5	32.54 $\pm$ 7.1	48.47 $\pm$ 7.1	42.55 $\pm$ 1.9 (+5.11)
Llama-4-Scout-17B-16E-Instruct										
Full Context	77.93 $\pm$ 5.6	0.00 $\pm$ 0.0	1.67 $\pm$ 1.5	60.45 $\pm$ 4.7	43.79 $\pm$ 4.7	67.91 $\pm$ 5.6	27.84 $\pm$ 6.8	39.64 $\pm$ 7.4	42.35 $\pm$ 6.9	40.18 $\pm$ 1.8
+ MemoBase	57.75 $\pm$ 6.6	5.62 $\pm$ 3.0	12.00 $\pm$ 3.8	67.41 $\pm$ 4.6	54.10 $\pm$ 4.7	27.61 $\pm$ 5.4	21.02 $\pm$ 6.0	47.34 $\pm$ 7.7	42.86 $\pm$ 6.6	37.30 $\pm$ 1.8 (-2.88)
+ Mem0	56.34 $\pm$ 6.6	3.21 $\pm$ 2.2	3.67 $\pm$ 2.2	66.17 $\pm$ 4.5	63.00 $\pm$ 4.7	45.90 $\pm$ 6.0	23.30 $\pm$ 6.0	51.48 $\pm$ 7.7	44.39 $\pm$ 6.9	39.72 $\pm$ 1.8 (-0.46)
+ Zep	71.36 $\pm$ 6.1	4.02 $\pm$ 2.4	7.00 $\pm$ 2.8	67.41 $\pm$ 4.5	52.69 $\pm$ 4.7	35.45 $\pm$ 6.0	27.84 $\pm$ 6.8	46.15 $\pm$ 7.7	46.43 $\pm$ 7.1	39.82 $\pm$ 1.8 (-0.36)
+ MemOS	67.61 $\pm$ 6.1	6.43 $\pm$ 3.0	11.33 $\pm$ 3.5	66.92 $\pm$ 4.5	64.17 $\pm$ 4.7	38.43 $\pm$ 6.0	23.86 $\pm$ 6.2	53.25 $\pm$ 7.7	50.00 $\pm$ 7.1	42.44 $\pm$ 1.9 (+2.27)
Gemini-3-Flash										
Full Context	97.65 $\pm$ 2.1	26.51 $\pm$ 5.4	45.00 $\pm$ 5.7	96.77 $\pm$ 1.7	98.36 $\pm$ 1.2	100.00 $\pm$ 0.0	67.05 $\pm$ 6.8	53.25 $\pm$ 7.7	68.88 $\pm$ 6.6	72.61 $\pm$ 1.6
+ MemoBase	56.34 $\pm$ 6.6	6.43 $\pm$ 3.0	17.67 $\pm$ 4.2	85.32 $\pm$ 3.5	91.10 $\pm$ 2.6	84.33 $\pm$ 4.3	38.07 $\pm$ 7.4	53.85 $\pm$ 7.7	69.39 $\pm$ 6.6	55.83 $\pm$ 1.8 (-16.78)
+ Mem0	56.34 $\pm$ 6.6	5.62 $\pm$ 3.0	2.67 $\pm$ 1.8	79.60 $\pm$ 3.9	84.54 $\pm$ 3.4	85.45 $\pm$ 4.3	36.93 $\pm$ 7.1	56.21 $\pm$ 7.4	61.73 $\pm$ 6.6	52.12 $\pm$ 1.8 (-20.48)
+ Zep	68.54 $\pm$ 6.1	6.02 $\pm$ 3.0	11.00 $\pm$ 3.5	85.82 $\pm$ 3.4	82.44 $\pm$ 3.5	78.36 $\pm$ 4.9	34.66 $\pm$ 6.8	60.95 $\pm$ 7.4	66.33 $\pm$ 6.6	54.90 $\pm$ 1.8 (-17.71)
+ MemOS	69.01 $\pm$ 6.1	10.84 $\pm$ 3.8	20.67 $\pm$ 4.7	81.84 $\pm$ 3.7	87.59 $\pm$ 3.0	90.67 $\pm$ 3.5	38.64 $\pm$ 7.1	62.72 $\pm$ 7.1	71.43 $\pm$ 6.4	59.27 $\pm$ 1.8 (-13.34)

Table 4: Main evaluation results on EverMemBench. "Full Context" uses the complete dialogue history; memory-augmented methods use only retrieved information. Best memory-augmented results per metric are bolded. Parenthesized values show accuracy change vs. Full Context. Gray subscripts are half-widths of 95% bootstrap CIs ($B=10,000$).

Model	Fine-Grained Recall			Memory Awareness		
	Single	Multi	Temp	Const	Proact	Update
GPT-4.1-mini	99.53	97.99	60.00	96.77	86.65	98.51
Llama-4-Scout	96.24	37.35	34.00	93.53	90.87	96.64
Gemini-3-Flash	98.14	88.37	54.33	99.26	98.12	99.23

Table 5: Oracle performance with ground-truth evidence spans provided directly to the model, isolating reasoning capability from retrieval quality.

the entire $\sim 1\text{M}$ -token dialogue history. Unless noted otherwise, all memory-augmented systems use GPT-4.1-mini as the answer model, allowing us to isolate retrieval effects.

Oracle Evaluation. To separate retrieval failures from reasoning limitations, we construct an oracle setting in which the ground-truth dialogue segments from which each answer must be synthesized are provided directly to the LLM. By bypassing retrieval entirely, oracle evaluation isolates each sub-dim’s reasoning demands over the provided evidence, which range from entity matching for Single-hop to lifecycle boundary disambiguation and date arithmetic for Temporal (Figure 15). Table 5 reports oracle performance. Profile Understanding is excluded from this setting because persona traits are implicit and distributed, leaving no discrete evidence spans that could serve as oracle input. Oracle performance thus defines an upper bound for any retrieval-only improvement, as it reflects model behavior when all relevant evidence is perfectly surfaced.

Evaluation Metrics. For *Fine-Grained Recall*, answers are concrete facts (names, numbers, timestamps) that may vary lexically while remaining semantically correct; we use LLM-as-a-judge [43] to assess equivalence. For *Memory Awareness* and *Profile Understanding*,

we use multiple-choice questions with carefully constructed distractors to prevent plausible fabrication by models without true memory access. This design ensures that high scores reflect genuine memory use rather than surface-level plausibility.

4.2 Fine-Grained Recall: Attribution and Time as Structural Bottlenecks

Multi-hop and temporal reasoning collapse under multi-party interleaving. As summarized in Table 4, all systems perform well on *Single-hop* recall (Gemini-3-Flash: 97.65%; memory systems: 55–83%), indicating that isolated fact retrieval is largely solved. However, *Multi-hop* accuracy drops sharply: Gemini-3-Flash falls to 26.51%, and the best memory-augmented system reaches only 18.88%. This is not a recall problem but an integration problem. In EverMemBench, relevant facts are distributed across speakers, groups, and days; answering correctly requires stitching together partial evidence that never co-occurs in a single exchange.

Oracle results in Table 5 confirm this diagnosis. When provided with ground-truth evidence, GPT-4.1-mini improves from 2.41% to 97.99% and Gemini-3-Flash from 26.51% to 88.37%, showing that strong models can reason correctly once attribution and retrieval are removed as bottlenecks. In contrast, LLaMA-4 reaches only 37.35% even under oracle conditions, frequently refusing to answer when evidence appears fragmented—highlighting a distinct failure mode rooted in conservative reasoning rather than retrieval.

The difficulty compounds as information spans more groups. Figure 4 shows that accuracy drops from 54.5% (single-group) to 33.6% (two groups) and 19.7% (three groups), a 64% relative decline. This degradation is consistent across models and memory configurations, demonstrating that cross-group attribution—not context length—is the dominant challenge.

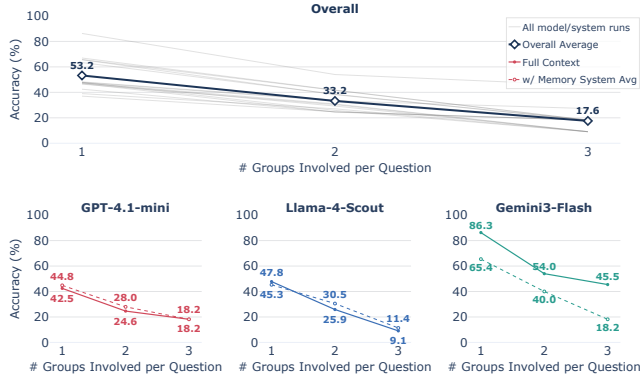


Figure 4: Accuracy by number of groups involved per question. Top: overall average across all settings. Bottom: breakdown by answer model.

Temporal reasoning remains unsolved. Temporal questions expose a complementary limitation. Across all systems, performance is low (Gemini-3-Flash: 45.00%; GPT-4.1-mini: 7.00%; memory systems: 2.67–21.00%). In realistic collaboration, decisions are revised, superseded, and finalized over time, and answering correctly requires reasoning over version semantics, not just timestamps. This difficulty persists even with oracle evidence: the best model reaches only 60% and LLaMA-4 only 34% (Table 5), because the provided evidence necessarily contains overlapping lifecycle signals such as premature completion announcements and archival statements from other speakers on adjacent dates, which the model must still disambiguate before computing durations. These results indicate a reasoning gap that current memory architectures cannot bridge, as they treat time as an ordering signal rather than a semantic construct that encodes event lifecycle stages such as initiation, revision, and completion.

4.3 Memory Awareness: Retrieval vs. Reasoning

Retrieval misses inferentially relevant evidence. Memory Awareness tasks are explicitly designed to test whether systems recognize when past information matters. Oracle results in Table 5 show that all models achieve 87–99% accuracy, confirming that reasoning capability is sufficient when relevant evidence is available. The difficulty lies in retrieving *the right* evidence.

Under full-context access, Gemini-3-Flash maintains near-oracle performance (97–100%), while GPT-4.1-mini and LLaMA-4 degrade substantially (25–63% and 43–68%). This gap reflects differences in long-context reasoning: weaker models struggle to locate relevant signals amid dense, multi-party dialogue.

Memory augmentation partially compensates for weaker models by filtering noise: GPT-4.1-mini improves to 29–76%. However, the same retrieval pipelines degrade Gemini-3-Flash (down to 76–90%) by discarding contextual cues the model could otherwise exploit. Even with identical retrieved evidence, performance diverges sharply—Gemini reaches 76–90%, while GPT-4.1-mini and LLaMA-4 reach only 29–76% and 35–73%. This reveals a dual bottleneck: retrieval fails to surface *inferentially relevant* information, and weaker answer models struggle to reason over sparse, indirect cues.

4.4 Profile Understanding: Emergent Patterns Resist Retrieval

Profile Understanding tasks probe whether systems can maintain consistent user models over time. Results in Table 4 show that *Style* is the most challenging subtask, with memory systems achieving only 11–58% and even Gemini-3-Flash reaching 67.05%. Unlike factual recall, communication style is an emergent pattern spanning many interactions; it cannot be recovered from isolated snippets. In contrast, *Skill* and *Role* achieve higher accuracy because they are partially inferable from individual task contexts and organizational structure. These results underscore a limitation of retrieval-centric memory: it excels at discrete facts but struggles with traits that are never stated and can only be captured by aggregating behavioral signals across the full interaction history.

Takeaway. Across all analyses, Figures 4 and Tables 4 and 5 jointly show that scaling context or retrieval alone is insufficient. Multi-party attribution, temporal revision, and inferential relevance introduce structural challenges that current memory paradigms do not address. EverMemBench surfaces these failures by design, providing a diagnostic benchmark that shifts evaluation from leaderboard comparisons to understanding *why* memory systems fail—and what next-generation architectures must change to succeed.

5 Conclusion

In this paper, we introduced EverMemBench, a high-realism benchmark for long-term conversational memory that reflects how LLMs are used in practice: as participants in sustained, multi-party collaboration where information is distributed across speakers and groups, revised over time, and implicitly shaped by roles and social context. Through a carefully controlled curation pipeline grounded in project timelines and public events, EverMemBench produces traceable dialogue logs and evidence-grounded QA items that stress memory challenges beyond context length, including attribution, temporal revision, and inferential relevance. Our empirical results show that many failures observed in deployed systems are structural rather than incidental: multi-hop reasoning collapses under multi-party attribution, temporal reasoning fails without explicit version semantics, and similarity-based retrieval struggles to surface implicitly relevant information even when reasoning capacity is sufficient.

These findings suggest clear directions for future research. First, memory architectures must move beyond flat or snippet-based storage toward representations that explicitly encode versioned state, episodic boundaries, and cross-group dependencies. Second, effective memory systems will need to integrate retrieval and reasoning more tightly, enabling models to recognize inferential relevance rather than relying solely on surface similarity. By decomposing memory competence into fine-grained recall, memory awareness, and profile understanding, EverMemBench provides a diagnostic foundation for systematically studying these challenges. We view EverMemBench as a cornerstone benchmark for the next generation of LLMs, one that enables principled progress toward structured, time-aware memory and socially grounded reasoning in realistic collaborative settings.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, et al. 2024. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2024).
- [2] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, et al. 2022. Improving language models by retrieving from trillions of tokens. *arXiv:2112.04426 [cs.CL]* <https://arxiv.org/abs/2112.04426>
- [3] Michelle Brachman, Amina El-Ashry, Casey Dugan, and Werner Geyer. 2025. Current and Future Use of Large Language Models for Knowledge Work. *arXiv:2503.16774 [cs.HC]* <https://arxiv.org/abs/2503.16774>
- [4] Jun Chen, Dannong Xu, Junjie Fei, Chun-Mei Feng, and Mohamed Elhoseiny. 2024. Document Haystacks: Vision-Language Reasoning Over Piles of 1000+ Documents. *arXiv:2411.16740 [cs.CV]* <https://arxiv.org/abs/2411.16740>
- [5] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshray Yadav. 2025. Memo: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv:2504.19413 [cs.CL]* <https://arxiv.org/abs/2504.19413>
- [6] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, et al. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. *arXiv:2507.06261 [cs.CL]* <https://arxiv.org/abs/2507.06261>
- [7] Ananya Ganesh, Martha Palmer, and Katharina von der Wense. 2023. A survey of challenges and methods in the computational modeling of multi-party dialog. In *Proceedings of the 5th Workshop on NLP for Conversational AI (NLP4ConvAI 2023)*, 140–154.
- [8] Google. 2025. Introducing Gemini 3 Flash: Benchmarks, Global Availability. <https://blog.google/products/gemini/gemini-3-flash/>.
- [9] Jia-Chen Gu, Chongyang Tao, and Zhen-Hua Ling. 2022. Who Says What to Whom: A Survey of Multi-Party Conversations. In *IJCAI*. 5486–5493.
- [10] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. REALM: Retrieval-Augmented Language Model Pre-Training. *arXiv:2002.08909 [cs.CL]* <https://arxiv.org/abs/2002.08909>
- [11] Junqing He, Liang Zhu, Rui Wang, Xi Wang, Reza Haffari, and Jiaying Zhang. 2024. MADial-Bench: Towards Real-world Evaluation of Memory-Augmented Dialogue Generation. *arXiv:2409.15240 [cs.CL]* <https://arxiv.org/abs/2409.15240>
- [12] Zhongtian Hu, Qi He, Ronghan Li, Meng Zhao, and Lifang Wang. 2025. Advancing Multi-Party Dialogue Framework with Speaker-ware Contrastive Learning. *arXiv:2501.11292 [cs.CL]* <https://arxiv.org/abs/2501.11292>
- [13] Zhaopei Huang, Qifeng Dai, Guozheng Wu, Xiaopeng Wu, Kehan Chen, Chuan Yu, Xubin Li, Tiezhen Ge, Wenxuan Wang, and Qin Jin. 2025. Mem-PAL: Towards Memory-based Personalized Dialogue Assistants for Long-term User-Agent Interaction. *arXiv:2511.13410 [cs.CL]* <https://arxiv.org/abs/2511.13410>
- [14] Bowen Jiang, Zhuoqun Hao, Young-Min Cho, Bryan Li, Yuan Yuan, Sihao Chen, Lyle Ungar, Camillo J. Taylor, and Dan Roth. 2025. Know Me, Respond to Me: Benchmarking LLMs for Dynamic User Profiling and Personalized Responses at Scale. *arXiv:2504.14225 [cs.CL]* <https://arxiv.org/abs/2504.14225>
- [15] Bowen Jiang, Yuan Yuan, Maohao Shen, Zhuoqun Hao, Zhangchen Xu, Zichen Chen, Ziyi Liu, Anvesh Rao Vijjini, Jiashu He, Hanchao Yu, Radha Poovendran, Gregory Wornell, Lyle Ungar, Dan Roth, Sihao Chen, and Camillo Jose Taylor. 2025. PersonaMem-v2: Towards Personalized Intelligence via Learning Implicit User Personas and Agentic Memory. *arXiv:2512.06688 [cs.CL]* <https://arxiv.org/abs/2512.06688>
- [16] Gregory Kamradt. 2023. Needle in a Haystack — Pressure Testing LLMs. https://github.com/gkamradt/LLMTest_NeedleInAHaystack.
- [17] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025. Memory OS of AI Agent. *arXiv:2506.06326 [cs.AI]* <https://arxiv.org/abs/2506.06326>
- [18] Mosh Levy, Alon Jacoby, and Yoav Goldberg. 2024. Same Task, More Tokens: The Impact of Input Length on the Reasoning Performance of Large Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 15339–15353. doi:10.18653/v1/2024.acl-long.818
- [19] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2021. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *arXiv:2005.11401 [cs.CL]* <https://arxiv.org/abs/2005.11401>
- [20] Zhiyu Li, Shichao Song, Hanyu Wang, Simin Niu, Ding Chen, Jiawei Yang, Chenyang Xi, Huayi Lai, Jihao Zhao, Yezhaohui Wang, Junpeng Ren, Zehao Lin, Jiahao Huo, Tianyi Chen, Kai Chen, Kehang Li, Zhiqiang Yin, Qingchen Yu, Bo Tang, Hongkang Yang, Zhi-Qin John Xu, and Feiyu Xiong. 2025. MemOS: An Operating System for Memory-Augmented Generation (MAG) in Large Language Models. *arXiv:2505.22101 [cs.CL]* <https://arxiv.org/abs/2505.22101>
- [21] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. doi:10.1162/tacl_a_00638
- [22] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. *arXiv:2402.17753 [cs.CL]* <https://arxiv.org/abs/2402.17753>
- [23] Memodb Team. 2025. MemoBase: User Profile-Based Long-Term Memory for AI Chatbot Applications. <https://github.com/memodb-io/memobase>.
- [24] Meta AI. 2025. The Llama 4 Herd: The Beginning of a New Era of Natively Multimodal AI Innovation. <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>.
- [25] Jiayan Nan, Wenquan Ma, Wenlong Wu, and Yize Chen. 2025. Nemori: Self-Organizing Agent Memory Inspired by Cognitive Science. *arXiv:2508.03341 [cs.AI]* <https://arxiv.org/abs/2508.03341>
- [26] Elliot Nelson, Georgios Kollias, Payel Das, Subhajit Chaudhury, and Soham Dan. 2024. Needle in the Haystack for Memory Based Large Language Models. *arXiv:2407.01437 [cs.CL]* <https://arxiv.org/abs/2407.01437>
- [27] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv:2501.13956 [cs.AI]* <https://arxiv.org/abs/2501.13956>
- [28] Rana Salama, Jason Cai, Michelle Yuan, Anna Currey, Monica Sunkara, Yi Zhang, and Yassine Benajiba. 2025. MemInsight: Autonomous Memory Augmentation for LLM Agents. *arXiv:2503.21760 [cs.CL]* <https://arxiv.org/abs/2503.21760>
- [29] Chris Samarinas and Hamed Zamani. 2024. ProCIS: A benchmark for proactive retrieval in conversations. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 830–840.
- [30] Sagar Sapkota, Mohammad Saqib Hasan, Mubarak Shah, and Santu Karmaker. 2025. Multi-Party Conversational Agents: A Survey. *arXiv:2505.18845 [cs.CL]* <https://arxiv.org/abs/2505.18845>
- [31] Haoran Tan, Zeyu Zhang, Chen Ma, Xu Chen, Quanyu Dai, and Zhenhua Dong. 2025. MemBench: Towards More Comprehensive Evaluation on the Memory of LLM-based Agents. *arXiv:2506.21605 [cs.CL]* <https://arxiv.org/abs/2506.21605>
- [32] Mohammad Tavakoli, Alireza Salemi, Carrie Ye, Mohamed Abdalla, Hamed Zamani, and J Ross Mitchell. 2025. Beyond a Million Tokens: Benchmarking and Enhancing Long-Term Memory in LLMs. *arXiv:2510.27246 [cs.CL]* <https://arxiv.org/abs/2510.27246>
- [33] Qingyue Wang, Yanhe Fu, Yanan Cao, Shuai Wang, Zhiliang Tian, and Liang Ding. 2025. Recursively summarizing enables long-term dialogue memory in large language models. *Neurocomputing* 639 (2025), 130193.
- [34] Bowen Wu, Wenqing Wang, Lihaoran Lihaoran, Yunhan Deng, Ying Li, Jingsong Yu, and Baoxun Wang. 2025. Interpersonal memory matters: A new task for proactive dialogue utilizing conversational history. In *Proceedings of the 29th Conference on Computational Natural Language Learning*. 47–67.
- [35] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. *arXiv:2410.10813 [cs.CL]* <https://arxiv.org/abs/2410.10813>
- [36] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic Memory for LLM Agents. *arXiv:2502.12110 [cs.CL]* <https://arxiv.org/abs/2502.12110>
- [37] Zhongyu Yang, Jun Chen, Dannong Xu, Junjie Fei, Xiaoqian Shen, Liangbing Zhao, Chun-Mei Feng, and Mohamed Elhoseiny. 2025. WikiAutoGen: Towards Multi-Modal Wikipedia-Style Article Generation. *arXiv:2503.19065 [cs.CV]* <https://arxiv.org/abs/2503.19065>
- [38] Zhongyu Yang, Yingfang Yuan, Xuanming Jiang, Baoyi An, and Wei Pang. 2025. InEx: Hallucination Mitigation via Introspection and Cross-Modal Multi-Agent Collaboration. *arXiv:2512.02981 [cs.CV]* <https://arxiv.org/abs/2512.02981>
- [39] Zihao Yi, Jiarui Ouyang, Zhe Xu, Yuwen Liu, Tianhao Liao, Haohao Luo, and Ying Shen. 2025. A survey on recent advances in llm-based multi-turn dialogue systems. *Comput. Surveys* 58, 6 (2025), 1–38.
- [40] Saizheng Zhang, Emily Dinan, Jack Urbanek, Arthur Szlam, Douwe Kiela, and Jason Weston. 2018. Personalizing Dialogue Agents: I have a dog, do you have pets too?. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Iryna Gurevych and Yusuke Miyao (Eds.). Association for Computational Linguistics, Melbourne, Australia, 2204–2213. doi:10.18653/v1/P18-1205
- [41] Siyan Zhao, Mingyi Hong, Yang Liu, Devamanyu Hazarika, and Kaixiang Lin. [n. d.]. Do LLMs Recognize Your Preferences? Evaluating Personalized Preference Following in LLMs. In *The Thirteenth International Conference on Learning Representations*.
- [42] Zheng Zhao, Clara Vania, Subhadeep Kayal, Naila Khan, Shay B Cohen, and Emine Yilmaz. 2025. PersonaLens: A Benchmark for Personalization Evaluation in Conversational AI Assistants. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 18023–18055. doi:10.18653/v1/2025.findings-acl.927
- [43] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *arXiv:2306.05685 [cs.CL]* <https://arxiv.org/abs/2306.05685>

A Data Statistics

A.1 Profile Distribution

Our benchmark comprises **170** unique employee profiles spanning a simulated enterprise organization. Each profile is characterized by demographic attributes, a skill portfolio, an 8-dimensional communication style, and a Big-Five personality trait vector.

Demographics. Figure 5 summarizes the demographic composition. Ages range from 23 to 48 (mean 30.8), with the majority (54.1%) in the 27–34 bracket. The gender split is 61.2% male and 38.8% female. Education levels include Bachelor’s (52.9%), Master’s (35.3%), and PhD (11.8%). The organizational hierarchy follows a five-level rank system: L1 (CEO, 0.6%), L2 (Directors, 2.9%), L3 (Senior Staff, 20.6%), L4 (Mid-level, 44.1%), and L5 (Junior, 31.8%), reflecting a realistic corporate pyramid.

Skill Profile. Each employee is assigned skills drawn from a pool of **104** unique competencies, with an average of 4.1 skills per member (range: 3–7). Skill assignments are rank-dependent:

- **Rank 1–2 (Executives):** 6–10 skills, including strategic management and cross-functional competencies.
- **Rank 3 (Senior Staff):** 4–6 skills, combining domain expertise with team coordination.
- **Rank 4–5 (Staff):** 3–5 skills, focused on role-specific technical or business competencies.

Each skill is annotated with a proficiency level: *strong* (27.2%), *medium* (47.9%), or *low* (24.9%). As shown in Figure 7, the most prevalent expert-level skills are Python and Java (each 13.9%), followed by Linux and Data Analysis (each 6.4%). Skills with less than 1% share are grouped into “Others.”

Communication Style. Each persona is assigned an 8-dimensional communication profile (Figure 6, left):

- **Formality:** Semi-formal (78.2%) / Casual (21.8%)
- **Verbosity:** Moderate (38.8%) / Concise (34.7%) / Detailed (26.5%)
- **Humor:** Minimal (50.0%) / Occasional (34.7%) / Frequent (15.3%)
- **Jargon Usage:** Technical (50.6%) / Balanced (30.6%) / Plain (18.2%)
- **Emoji Usage:** Rare (54.7%) / Occasional (28.2%) / Frequent (17.1%)
- **Directness:** Balanced (57.6%) / Direct (41.2%) / Indirect (1.2%)
- **Warmth:** Friendly (81.8%) / Neutral (11.8%) / Warm (6.5%)
- **Questioning Style:** Probing (47.6%) / Clarifying (44.7%) / Accepting (7.6%)

Assignments are conditioned on role expectations: executives tend toward formal, direct, and neutral styles, while technical staff favor concise, technical, and minimal-humor communication.

Personality Traits. Big-Five personality traits are assigned at three levels (High/Medium/Low), as visualized in Figure 6 (right). Openness skews high (44.7% High, 54.1% Medium), Extraversion skews low (51.8% Low), and Agreeableness is predominantly medium (77.6%). Management-level employees (L1–L2) exhibit markedly higher Openness and Extraversion compared to staff (L3–L5), consistent with leadership role expectations.

Topic	Participants	Messages	Sub-tasks	QA
Technology	39	10,222	436	488
Operations	36	10,106	408	471
Marketing	45	10,638	429	467
Financial Services	32	10,003	423	481
Governance	46	10,054	424	493
Total	170	51,023	2,120	2,400

Table 6: Dialogue statistics across five project topics. Sub-tasks denote the number of distinct work tasks respectively.

Overall Analysis. These distributions are intentionally skewed to mirror realistic enterprise communication norms—semi-formal tone and friendly warmth predominate in professional group chats, while the rank pyramid (75.9% at L4–L5) reflects typical corporate hierarchies. Although certain individual dimensions are concentrated (e.g., Semi-formal 78.2%, Friendly 81.8%), the impact on benchmark validity is limited: each persona is characterized by a *combination* of all eight style dimensions, so even personas sharing the same value on one axis differ substantially along others—the combinatorial space ensures diverse and distinguishable style fingerprints across the 170 employees. Moreover, as shown in Figure 6, Management and Staff profiles diverge clearly on Formality, Warmth, and Questioning, confirming meaningful inter-role variation. Finally, Style accounts for only 7.3% of all QA items (176/2,400), and our 2×2 fact-vs-style adversarial distractor design prevents systems from exploiting distributional priors.

A.2 Dialogue Distribution

The dialogue corpus spans **5 topics**, each simulating approximately one year (Jan–Dec 2025) of enterprise group-chat communication. As summarized in Table 6, the dataset contains **51,023 messages** comprising over **2.1 million words**, with a consistent average message length of 41.5 words across all topics.

Multi-Theme Structure. Each topic is organized into **3 concurrent project groups**, with 32–46 participants communicating over 250–255 workdays. Conversations revolve around a rich set of **834–966 distinct themes** per topic (4,556 in total), composed of two sources: (1) *work tasks* (408–436 per topic), representing ongoing project activities such as feature development, code reviews, and design discussions; and (2) *injected news events* (405–543 per topic), simulating real-world information that employees discuss organically—covering industry trends, policy updates, and current affairs. This dual-source design ensures that the dialogue corpus captures both structured work-related memory and loosely-structured world-knowledge discussions, posing a realistic challenge for memory systems that must distinguish and retrieve from heterogeneous conversational threads.

QA Benchmark. Each topic is accompanied by a curated set of question–answer pairs, totaling **2,400** across all five topics after filtering. QA pairs are organized into **three evaluation dimensions** encompassing **nine sub-tasks**:

- **Fine-grained Recall** (762 pairs, 31.8%): Single-hop Retrieval (213), Multi-hop Trajectory (249), and Temporal Duration (300).

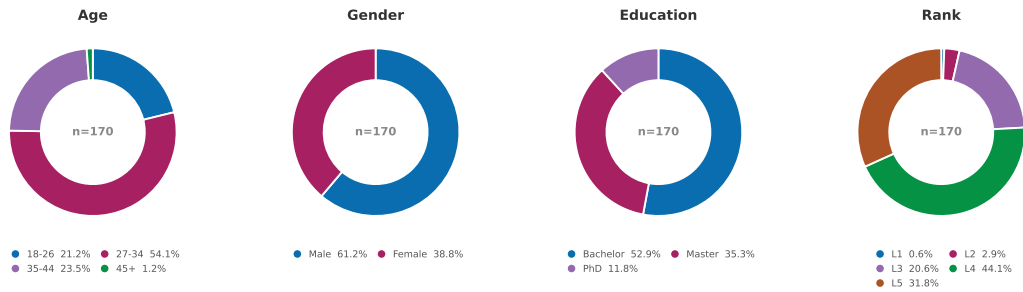


Figure 5: Demographic distributions of the 170 simulated user profiles across four dimensions: age, gender, education level, and organizational rank.

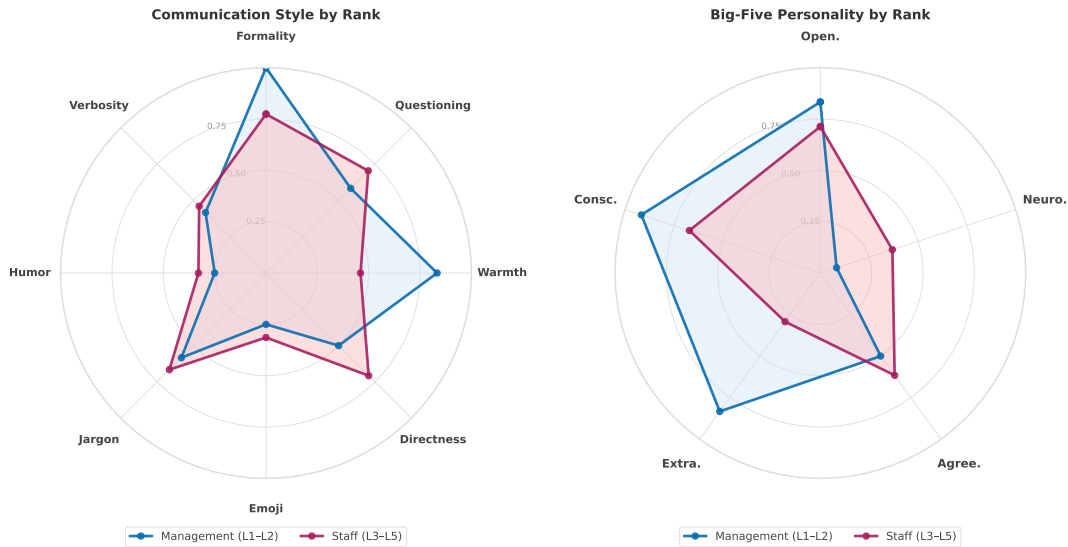


Figure 6: Radar charts comparing communication style (left) and Big-Five personality traits (right) between management (L1-L2) and staff (L3-L5) groups.

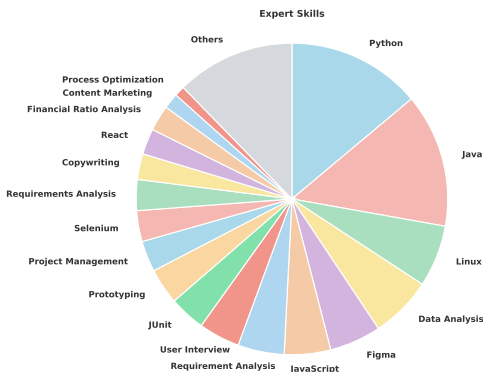


Figure 7: Distribution of expert skills among simulated user profiles. The top-18 skills are shown individually; skills representing less than 1% of the total are grouped into “Others.”

- **Memory Awareness** (1,097 pairs, 45.7%): Constraint (402), Proactivity (427), and Update (268).
- **Profile Understanding** (541 pairs, 22.5%): Style (176), Skill (169), and Role (196).

Each QA pair includes an average of 6.7 evidence references pointing to specific messages in the dialogue history, enabling fine-grained evaluation of both retrieval precision and comprehension-synthesis capability. Fine-grained Recall questions adopt an open-ended format because each question targets a well-defined entity (a specific link, date, or person), making the gold answer unambiguous and straightforward to verify. Memory Awareness and Profile Understanding questions, by contrast, assess generalization and contextual understanding of stored memories—capabilities whose correct answers are more nuanced and harder to judge automatically. To minimize variance introduced by LLM-as-a-Judge scoring, we adopt a multiple-choice format (A/B/C/D) for these two dimensions, where both questions and distractors are carefully crafted to prevent information leakage from the question stem and to ensure that every wrong option remains plausible rather than serving as a strawman (see §B.3 for distractor design details).

B Data Construction Details

B.1 Blueprint Generation

The blueprint pipeline proceeds through six stages, each followed by automated validation. Below we detail each stage.

Stage 1: Project Topic Generation. We begin by generating five project topics spanning diverse enterprise domains—Technology, Operations, Marketing, Financial Services, and Governance—each characterized by a sector-specific context, three to five core challenges, and a set of expected stakeholder roles. Gemini-2.5-Pro first produces a broad pool of candidates; to steer diversity, we provide contrasting examples: desirable topics cross sector boundaries (smart manufacturing, cross-border e-commerce, telemedicine), while near-duplicate clusters (e.g., “intelligent customer service / intelligent recommendation / intelligent analytics”) are explicitly excluded. Human annotators then screen, merge, and refine the candidates, adjusting scope and resolving overlaps until each surviving topic can sustain a year-long simulated development cycle.

Stage 2: Sub-Project Decomposition. Each topic is decomposed into three sub-projects, yielding $5 \times 3 = 15$ distinct units. A Financial Services project, for example, might split into a Risk Assessment Engine, a Transaction Processing System, and a Compliance Reporting Platform. The key constraint is *mutual exclusivity*: sub-projects must be independently executable, so we forbid parent–child overlaps (e.g., “develop the manufacturing execution system” alongside “develop its production scheduling module”). A post-generation validator verifies that no sub-project subsumes another. This structure creates natural cross-group information flow—decisions in one sub-project (say, a risk scoring methodology) constrain design choices in another (transaction validation rules)—while keeping each group chat channel self-contained with its own team roster and task backlog.

Stage 3: Team Member Selection. For each sub-project we assemble a team from the shared pool of 170 employees across seven departments (1 Executive, 5 Managers, 164 Staff). The LLM receives the complete employee registry—skill profiles s_e with proficiency levels and 8-dimensional communication styles c_e —and selects members by balancing skill–task alignment, communication complementarity, and hierarchical coverage. A validator then enforces that every team spans all three ranks and falls within the configured size range. On average, each project involves 37.6 participants.

Stage 4: Communication Style Adaptation. Because communication behavior shifts with team context, we adjust each member’s style profile after assembly. Given the team’s rank distribution and size, the LLM shifts individual dimensions by up to two levels and records the rationale; original profiles are preserved.

Stage 5: Subtask Generation and Sequencing. For each sub-project we generate a set of subtasks $\mathcal{T}_{p,j}$ organized into six sequential development phases: Strategy & Planning, Requirements & Design, System Architecture & Tech Selection, Development & Integration, Testing & QA, and Deployment & Operations.

The main challenge here is granularity. A coarse task like “conduct requirements research” is insufficient; it must be broken into

concrete units such as “survey target user personas,” “collect competitor feature lists,” and “interview internal business departments.” Likewise, “design the database” becomes per-table schema design, indexing strategy, and documentation. We supply extensive positive and negative examples to calibrate the LLM’s decomposition depth. Each resulting subtask specifies a concrete deliverable, one to three required skills drawn from the skill universe, a development phase, and a dependency type. Tasks are topologically sorted so that architectural decisions precede implementation, API contracts precede integration, and testing follows feature completion. Deadlines are spread across the full calendar year, with early phases concentrated in Q1–Q2 and later phases in Q3–Q4.

Stage 6: Subtask-to-Member Assignment. Each subtask is assigned to exactly one team member. Given the ordered subtask list and each member’s skill profile, the LLM matches tasks to people by considering proficiency alignment (strong > medium > low), workload balance, and role appropriateness (Executive handles strategy; Managers oversee architecture and coordination; Staff execute implementation and testing). Every assignment comes with a short natural-language rationale that serves as an audit trail. The validator ensures that every member receives at least five subtasks—guaranteeing meaningful dialogue participation—that no subtask is left unassigned, and that temporal feasibility constraints are met.

Cross-Group Dependency Injection. After individual blueprints are complete, we inject cross-project dependencies to create the multi-group structure central to EverMemBench. Within each project the three sub-projects already share personnel (especially Managers and the Executive) and technical decisions, producing natural coupling. We formalize three types of dependency: *data contracts*, where one sub-project defines a schema that another consumes and must therefore wait for; *shared infrastructure*, where multiple sub-projects rely on a common component (e.g., an authentication service) managed by one team; and *policy decisions*, where a technical standard adopted in one sub-project constrains choices elsewhere (e.g., the state management library or deployment platform). Each dependency is recorded as a tuple $(P_i, P_j, \text{type, artifact})$, enabling downstream dialogue generation to reference cross-group decisions and supporting questions that require multi-group evidence chaining.

Blueprint Validation. Beyond the per-stage validators described above, a final global check ensures end-to-end consistency before dialogue generation begins. We verify five invariants: (1) *completeness*—every subtask has an assigned owner, a deadline, and required skills; (2) *temporal consistency*—no task depends on one scheduled later; (3) *skill coverage*—every required skill appears in at least one team member’s profile at medium proficiency or above; (4) *load balance*—no employee carries an unreasonable number of concurrent tasks in any 30-day window; and (5) *role coherence*—strategic tasks are assigned to Executives or Managers, not junior staff. Any failure triggers targeted regeneration of the offending stage.

The resulting blueprints serve as the executable specification for dialogue generation: every conversation references specific subtasks, respects the defined dependencies, and exhibits role-appropriate communication patterns grounded in the blueprint’s organizational structure.

B.2 Conversation Generation

Conversation generation transforms the static blueprints into dynamic, day-by-day multi-party dialogues spanning a full simulated year ($D = 365$ days). As introduced in the main text, each sub-project maintains its own group chat channel, and each day’s dialogue is generated in a single forward pass conditioned on hierarchical summaries and persona profiles. Dialogues are generated only on workdays; weekends contain no conversations, mirroring the schedule of a real enterprise. This appendix expands on the task scheduling pipeline that bridges blueprints to daily generation, the three-layer summarization mechanism, and the multi-level quality assurance system.

Task Schedule Preparation. Before dialogue generation begins, the blueprint’s subtask assignments must be converted into a concrete daily schedule that determines which tasks are active on each simulated day. This proceeds in three steps. First, we extract all subtasks from the completed blueprints, adjust deadlines that fall on weekends to the nearest following workday, and deduplicate tasks that appear in overlapping sub-project assignments. Second, Gemini-2.5-Pro [6] estimates the working-day duration of each task, taking into account its complexity, phase, and inter-task dependencies. Third, a backward-scheduling algorithm computes each task’s start date by subtracting the estimated duration from its deadline, automatically resolving conflicts when a predecessor’s deadline falls after a dependent task’s computed start date. The resulting schedule classifies every task on each workday into one of three categories: *starting* (first day of work), *ongoing* (in progress but not yet due), and *ending* (deadline reached). This three-way partition drives the dialogue generator’s expectations about what each day’s conversation should contain (new task kickoffs, progress updates, or completion announcements), and is enforced by downstream validation.

Daily Dialogue Synthesis. For each workday d , the dialogue generator receives five categories of input: (i) the day’s task schedule with its starting/ongoing/ending classification and the identities of each task’s assignees, (ii) the most recent weekly summary $W_{p,j}^{(d)}$, (iii) the current monthly summary $M_{p,j}^{(d)}$, (iv) outstanding leader instructions $L_{p,j}^{(d)}$ from the past 30 days, and (v) the full persona profiles $\{\pi_e\}_{e \in \mathcal{E}_{p,j}}$ of all group members, including their team-adapted communication styles from Stage 4 of the blueprint pipeline.

The generator produces a structured multi-party dialogue where each utterance is attributed to a specific speaker, timestamped within work hours (09:00–18:00), and bound to the task identifiers it discusses. Two constraints are particularly important. First, every task assignee must participate in the discussion of their assigned tasks, since a passive team member who never speaks about their own work would be unrealistic and would deprive downstream QA generation of evidence material. Second, strict deadline discipline is enforced: only tasks whose deadline falls on the current day may be marked as completed, while ongoing tasks may report progress but not closure. This constraint is essential for creating the temporal revision patterns that our evaluation probes; without it, premature completion declarations would collapse the version-tracking complexity needed for temporal reasoning questions.

Hierarchical Summarization. Generating coherent dialogue over a full calendar year under LLM context limits requires compressing history without losing the structural cues that support temporal reasoning. As formalized in the main text, we employ three complementary summarization layers.

The *weekly summary* $W_{p,j}^{(d)}$ is generated at the end of each week from the preceding days’ dialogues. It organizes progress by task, recording dated milestones in chronological order and noting planned next steps. This provides the dialogue generator with fine-grained recent context (what was decided earlier in the week, what remains blocked, what is expected next), enabling natural continuity across week boundaries.

The *monthly summary* $M_{p,j}^{(d)}$ is generated on the last workday of each month from all dialogues and task records within that period. It groups information by project, listing completed tasks with their completion dates, in-progress items with current status, key milestones, and accumulated factual details such as budgets, team sizes, and technical decisions. Where the weekly summary captures operational momentum, the monthly summary provides the long-range backdrop that prevents the dialogue from contradicting decisions made weeks earlier.

The *leader instruction* $log L_{p,j}^{(d)}$ extracts and retains directives from senior personnel (Executive and Managers) over a rolling 30-day window. Each instruction is categorized as a strategic decision, an operational directive, or a methodological guidance, and is linked to the affected tasks. This mechanism ensures that downstream conversations comply with, not merely mention, the decisions of leadership, creating the policy-adherence patterns targeted by our Memory Awareness evaluation dimension.

Multi-Level Quality Assurance. Each day’s generated dialogue passes through four sequential validation levels before being accepted, extending the produce–verify philosophy introduced in the blueprint pipeline to the more complex domain of natural-language dialogue.

Level 1 applies deterministic programmatic rules: timestamps must fall within work hours, speakers must be authorized members of the group they appear in, group names must match the project structure exactly, and raw task identifiers must not leak into dialogue text (task references are maintained through structured metadata bindings). These checks are instantaneous and catch mechanical formatting errors before more expensive semantic analysis begins.

Level 2 uses an LLM to verify task-completion semantics against the day’s schedule. Starting tasks must show evidence of initiation; ending tasks must show evidence of completion; and critically, no task may exhibit premature completion. The checker draws a semantic distinction between forward-looking statements (“I will finish this today,” “submitting version 1.0 for review”) and genuine closure markers (“this task is now complete, deliverables archived”), since conflating the two would undermine the temporal integrity of the data.

Level 3 evaluates logical coherence: an LLM examines the conversation for internal self-contradictions, inconsistencies with information established in weekly and monthly summaries, and violations

of recorded leader directives. This check operates with a deliberately lenient threshold, flagging only substantive errors that would make QA items unanswerable or ambiguous.

Level 4 verifies communication quality against persona profiles: appropriate forms of address (subordinates using formal titles for superiors), absence of self-referencing errors (speakers unnaturally mentioning their own names), alignment with each speaker’s 8-dimensional communication style, and role-appropriate expertise in technical discussions.

Bounded Regeneration. When validation fails at any level, the system attempts targeted repair rather than regenerating the entire day’s dialogue from scratch. A dedicated fix agent receives the flagged dialogue along with a structured conflict report detailing each violation by category and severity. The agent applies minimal, conservative edits, preferring deletion over modification to avoid introducing new errors, such as removing utterances from unauthorized speakers, adjusting out-of-range timestamps, converting premature completion claims into progress statements, and correcting task-identifier bindings. The repaired dialogue is then re-checked through the full four-level pipeline. If validation still fails after two repair attempts, the date is flagged for manual review. All repairs are logged with before-and-after diffs to maintain a complete audit trail and support post-hoc analysis of common generation failure modes.

Cross-Group Coherence. Within each project, the three sub-project group chats share personnel, particularly Managers and the Executive, who participate in multiple channels simultaneously. The generation pipeline maintains coherence across groups by ensuring that shared members cannot be speaking in two groups at the exact same timestamp, and that technical decisions referenced across groups remain consistent. When the dialogue generator encounters a task that depends on a cross-group artifact (a data contract, shared infrastructure component, or policy decision injected during the blueprint phase), it conditions the conversation on the relevant cross-group context so that references to external decisions are accurate and temporally appropriate. This mechanism is what makes multi-hop, cross-group QA items possible: the evidence trail genuinely spans multiple group chats rather than being artificially duplicated.

B.3 Q&A Generation

We synthesize 2,400 QAE triples (q, a, e) through three specialized pipelines, one per evaluation dimension. All three pipelines share a common principle we call the *missing-key rule*: each question provides a scenario (the “lock”) whose resolution depends on specific evidence buried in the dialogue history (the “key”), and the question itself must never leak the key. This principle guides both question design and distractor construction across all dimensions. Below we detail the generation strategy for each dimension.

Fine-grained Recall. This dimension evaluates precise retrieval of entities, timestamps, and events from dense multi-turn discussions. We organize generation into sub-branches of increasing difficulty. *Single-hop retrieval* targets direct entity grounding: extracting a specific deliverable link, a stated deadline, or a named assignee from a particular discussion thread. *Duration and interval* questions

require computing time spans, either the working-day length of a single task or the gap between two events (e.g., from one task’s completion to the next task’s initiation by the same person). *Existence checks* include both positive cases (verifying that a stated event did occur) and negative cases designed to test hallucination resistance, where we construct near-miss scenarios referencing plausible but non-existent events. *Multi-hop retrieval* requires chaining evidence across speakers or groups: identifying a person through a compositional constraint (e.g., “the colleague responsible for writing the Swagger documentation for authentication”), then tracing that person’s timeline to a subsequent event. When the base narrative lacks the specific evidence structure needed for a multi-hop chain, we employ controlled implantation via *field-bridge decoupling*: a target reasoning chain R_0 is decomposed into a bridge segment R_2 that maps a scenario to an intermediate identifier and a dictionary segment R_3 that resolves the identifier to a concrete solution. These segments are embedded into the dialogue history as plausible meeting notes or data-dictionary entries authored by real project members; seeing R_3 alone reveals no connection to the original scenario, so the solver must retrieve R_2 to complete the chain. To create a realistic difficulty gradient, we additionally extract a decoy rule R_1 from existing dialogue, representing a general default procedure that the question naturally evokes. The question references R_1 ’s context but includes a subtle special condition that triggers the override path through R_2 and R_3 , forcing the system to recognize that the general rule does not apply and to follow the multi-hop chain instead.

For all sub-branches, we apply *structure mining*: an LLM agent traverses the blueprint and dialogue structure to identify naturally occurring retrieval targets, such as task completion markers, cross-group decision references, and personnel role bindings. To ensure diversity, we use embedding-based similarity filtering to prevent semantically redundant questions from clustering around the same dialogue segments.

Memory Awareness. This dimension tests whether the system can mobilize past information to solve new problems, not merely recall it. Generation follows a five-step pipeline for each QA item. In the first step, the generator selects one to four dialogue snippets as evidence, focusing on segments that contain logical anchors: constraint definitions, contradiction points, or high-stakes directives. In the second step, a verification agent checks factual accuracy, logical soundness, and evidence completeness. In the third step, the generator produces the question, the correct answer, and three distractors. In the fourth step, a chain-of-thought auditor checks whether the question is answerable without evidence (which would indicate information leakage), whether all options appear equally authoritative, and whether length imbalance reveals the correct answer. In the fifth step, difficulty is upgraded through adversarial perturbations: keyword substitution replaces explicit technical terms with functional descriptions to force semantic rather than lexical retrieval; parameter removal strips explicit constraint mentions from the question so that the solver must infer them from evidence; and honey-trap options are added that satisfy common sense but violate specific rules established in the dialogue.

The three sub-tasks target distinct cognitive operations. *Constraint* questions present novel scenarios that require applying

implicit rules extracted from past discussions (e.g., recognizing that a data-contract owner, not the downstream consumer, must approve schema changes). *Proactivity* questions simulate situations where the user issues an instruction that conflicts with a previously established hard rule; the system must detect the violation and provide a reminder rather than blindly executing. *Updating* questions require resolving chronological precedence: initial specifications are later revised, and the system must identify the current valid state rather than returning stale information.

Profile Understanding. This dimension examines whether the system can aggregate distributed signals into stable user models. We employ a 2×2 fact-versus-style matrix to construct distractors that disentangle factual correctness from persona alignment. For each question, we generate four options: the correct answer (fact correct, style correct), F1 (fact correct, style wrong), F2 (fact wrong, style correct), and FF (fact wrong, style wrong). The critical distractor is F2, which perfectly mimics the target person’s communication style while containing fabricated factual content. F2 is deliberately made 10–20% longer than the correct answer to exploit the tendency of language models to prefer more detailed options. FF reuses F2’s sentence structure with only the style expressions swapped, preventing it from being dismissible as a strawman.

Factual errors in F2 and FF follow five misleading strategies: quoting real names and timestamps from the evidence but fabricating conclusions (half-true-half-false); attributing one person’s work to another (role misattribution); reversing temporal states such as “completed” versus “in progress” (temporal reversal); inverting cause-effect relationships (causal inversion); and subtly modifying quantities or degrees (degree manipulation). All fabricated facts must remain professionally plausible, representing alternative reasonable decisions rather than obvious technical absurdities.

The three sub-tasks probe different facets of user modeling. *Style* questions ask the system to draft a reply on behalf of a specific person, where the correct option matches both the factual record and the person’s characteristic communication register (formality, emoji usage, humor). *Skill* questions require recommending a technical solution that aligns with both the speaker’s personal expertise and the team’s ratified technology stack. *Role* questions test whether improvement suggestions are framed from the speaker’s professional perspective (e.g., a developer proposing pipeline reviews rather than test-case documentation).

B.4 Q&A Quality Control

Quality control enforces three properties: non-triviality (items cannot be solved without context), soundness (items can be solved with the designated evidence), and uniqueness (items cannot be solved from unintended evidence segments). We implement these through a three-phase filter that deliberately favors conservative retention.

B.4.1 Blind Test. The blind test eliminates items solvable through parametric knowledge or annotation artifacts. Each QA item is stripped of all dialogue context and evidence, leaving only the question and answer options, and submitted in parallel to four frontier LLMs (Gemini-2.5-Pro, Claude Sonnet 4.5, GPT-5.1, and

Qwen3-235B). Each model returns a predicted answer, a confidence score, and a reasoning chain.

We classify results into four scenarios. *Scenario A* (serious leakage): a majority of models answer correctly with high confidence, indicating that the question is solvable from world knowledge alone; these items are rejected. *Scenario B* (random guessing): answers are dispersed with low confidence, confirming that context is required; these items are retained. *Scenario C* (strong distractors): a majority of models answer incorrectly, suggesting that the distractors are effective; these are marked as high-quality items. *Scenario D* (premium difficulty): only one or two models answer correctly with sophisticated reasoning, indicating that the question is challenging but fair. Only Scenario A items are automatically rejected; all others proceed to Phase II.

B.4.2 Evidence Grounding. Evidence grounding verifies that each item is both answerable from its designated evidence and not answerable from any other dialogue segment. We partition the full dialogue corpus C_p into segments aligned with subtask boundaries from the blueprint. For each QA item, the segment containing its evidence is designated as S^+ (positive context) and all remaining segments as S^- (negative contexts).

The *sufficiency test* provides an LLM with the question, options, and S^+ , requiring it to derive the correct answer through explicit reasoning. The model must extract specific dialogue citations (speaker, timestamp, content) supporting its choice. If the model fails to answer correctly or selects “cannot determine,” the item is flagged as insufficiently grounded. The test also records which portions of the original evidence the model actually used, providing an R-coverage metric that measures how much of the annotated evidence is necessary versus redundant.

The *uniqueness test* checks whether any negative segment accidentally supports the correct answer. For each $S^- \in S \setminus \{S^+\}$, we conduct a two-phase probe. In the first phase (fast screening), the model is forced to select among A/B/C/D without a “cannot determine” option; if it selects incorrectly, the segment is cleared. If it selects correctly, a second phase provides the “cannot determine” option and requests detailed reasoning. Items where the model still selects the correct answer in the second phase are flagged as having uniqueness issues, since the evidence for that answer exists in an unintended location.

Items that fail sufficiency are rejected outright. Items with uniqueness warnings are escalated to human review, where annotators determine whether the unintended evidence path represents a genuine alternative or a false positive from the LLM’s reasoning.

B.4.3 Human Audit. The final phase addresses residual logical and pragmatic issues that automated filters cannot reliably detect. Items are prioritized by risk level: those flagged by Phase II receive priority review, items that passed both automated phases receive spot checks. Human reviewers evaluate four dimensions: question reasonability (does the question follow natural human inquiry patterns?), answer rigor (is the correct answer unambiguous and fully evidence-based?), distractor deceptiveness (are wrong options genuinely challenging rather than obviously implausible?), and evidence sufficiency (does the annotated evidence fully support the reasoning chain?). Items that fail any dimension are either revised and re-tested through the full pipeline or rejected entirely.

This conservative approach ensures that benchmark failures can be confidently attributed to system limitations rather than annotation noise.

B.4.4 Post-Hoc Validation. As a final check, we verify that the curated benchmark remains non-trivial after all filtering stages. We evaluate GPT-4.1-Mini in a *zero-context* setting: the model receives only the question and answer options with no dialogue history or retrieved memories. As shown in Table 7, performance across all nine sub-tasks remains at or below chance level (25% for four-way multiple choice), with Multi-Hop retrieval as low as 2.01%. This end-to-end result complements the construction-time blind test (§B.4.1): whereas the blind test filters out items answerable by frontier LLMs *before* inclusion, the zero-context baseline confirms that the final item set cannot be solved without access to the underlying conversation corpus.

Model	Fine-Grained Recall			Memory Awareness			Profile Understanding		
	Single	Multi	Temp.	Const.	Proac.	Upd.	Style	Skill	Role
GPT-4.1-Mini	15.02	2.01	12.67	11.69	10.07	7.84	17.61	26.04	26.53

Table 7: Zero-context performance of GPT-4.1-Mini without any memory access. All scores remain at or below chance level, confirming that questions require genuine memory retrieval.

C Evaluation Details

C.1 Answer Prompts

We use separate prompts for multiple-choice and open-ended questions. For memory-augmented systems, retrieved memories serve as context (Figures 9–8); for the full-context LLM baseline, the complete dialogue transcript is provided directly (Figures 11–10).

C.2 LLM-as-a-Judge Prompt

We adopt an LLM-as-a-Judge protocol for open-ended answer evaluation. The judge receives the question, gold answer, and generated answer, and returns a binary CORRECT/WRONG label with a one-sentence rationale (Figure 12).

C.3 LLM-as-Judge Reliability

We randomly selected 30 non-overlapping open Q&A pairs from EverMemBench, and generated model answers for each question. We recruited annotators via Prolific. For each Q&A pair, five independent human evaluators judged whether the generated answer was correct given the question and the reference answer. All participants provided informed consent via the platform interface and were compensated at approximately \$12.00/hour, consistent with fair-pay guidelines for academic research and above local minimum wage standards. Table 8 shows strong agreement between the LLM-as-judge protocol and human annotations. These results suggest that LLM-as-Judge achieves human-level reliability for answer verification, enabling evaluation that is rigorous, reproducible, and cost-efficient.

System	Cohen’s κ	95% CI	Accuracy	Pearson r
MemOS	0.927	[0.756, 1.000]	96.7%	0.929
MemoBase	0.930	[0.756, 1.000]	96.7%	0.932

Table 8: Reliability matrix for LLM-as-Judge.

D Additional Experimental Results

D.1 Performance across Different Topics

System	T1	T2	T3	T4	T5
LLM	38.98	35.43	37.79	35.98	38.90
Zep	40.00	39.15	36.37	43.93	40.50
Mem0	35.97	30.02	42.49	37.23	39.99
MemOS	40.73	41.05	42.03	41.58	43.21
MemoBase	32.51	28.57	31.25	30.83	33.26

Table 9: Performance across five project domains (T1–T5: Technology, Operations, Marketing, Financial Services, Governance). Bold indicates the best system for each topic.

Table 9 reports performance across five project domains. System rankings are largely stable: MemOS leads on three of five topics (T1, T2, T5) and places second on the remaining two, while MemoBase ranks last on all five domains, indicating that capability gaps are fundamental rather than domain-dependent. Cross-domain average accuracy ranges from 34.8% (T2) to 39.2% (T5), confirming that no single domain is systematically easier or harder. Cross-domain stability varies notably across systems: MemOS shows the smallest performance spread ($\Delta=2.5$ pp), while Mem0 exhibits the largest ($\Delta=12.5$ pp, dropping from 42.49% on T3 to 30.02% on T2), suggesting that some retrieval strategies are more sensitive to domain-specific information structures than others.

D.2 Performance Across Different Groups Numbers

QAR Group Count	1	2	3
Answer Model: GPT-4.1-mini			
llm	0.4247	0.2463	0.1818
zep	0.4743	0.2925	0.0909
mem0	0.4694	0.2449	0.1818
memos	0.4791	0.3333	0.2727
memobase	0.3702	0.2476	0.1818
Answer Model: Llama4-Scout			
llm	0.4779	0.2585	0.0909
zep	0.4646	0.3034	0.0909
mem0	0.4743	0.3088	0.0909
memos	0.4822	0.3388	0.1818
memobase	0.3902	0.2707	0.0909
Answer Model: Gemini-3-Flash			
llm	0.8627	0.5401	0.4545
zep	0.6576	0.4163	0.1818
mem0	0.6340	0.3850	0.1818
memos	0.6697	0.4177	0.1818
memobase	0.6564	0.3823	0.1818

Table 10: Accuracy by Grouping across Answer Models

Table 10 further breaks down accuracy by the number of cross-group hops required, under three answer models. The cross-group degradation pattern observed is consistent across all answer models: accuracy drops sharply from single-group to three-group questions regardless of backbone capability.

Memory-Augmented Answer Prompt — Open-Ended

You are an intelligent memory assistant tasked with retrieving accurate information from conversation memories. You will be given retrieved memories from a multi-person group chat and one open-ended question. Your task is to answer the question using **ONLY** the information in the memories.

INSTRUCTIONS:

1. Carefully analyze all provided memories from the group chat.
2. Pay special attention to timestamps to determine when events occurred.
3. If the question asks about a specific event or fact, look for direct evidence in the memories.
4. If memories contain contradictory information, prioritize the most recent memory.
5. If the question involves time references (like "last year", "two months ago"), calculate the actual date based on the memory's timestamp.
6. Always convert relative time references to specific dates, months, or years in your answer.
7. Pay attention to who said what — the memories may involve multiple participants.
8. The answer should be concise and specific (under 5–6 words when possible).
9. Do NOT output any reasoning steps, explanations, or extra text beyond the final answer.

APPROACH (Think step by step internally):

1. Examine all memories that contain information related to the question.
2. Examine timestamps and content carefully.
3. Look for explicit mentions of dates, times, locations, or events that answer the question.
4. If the answer requires calculation (e.g., converting relative time references), do so.
5. Formulate a precise, concise answer based solely on the evidence in the memories.

Output format (must be followed exactly):

Output **ONLY** the answer text

[MEMORIES] {context}

[QUESTION] {question}

Figure 8: Memory-augmented answer prompt for open-ended questions.

Memory-Augmented Answer Prompt — Multiple-Choice

You are a rigorous question-answering assistant. You will be given retrieved memories from a multi-person group chat and one multiple-choice question with four options. Your task is to choose the single best answer (A/B/C/D) based **ONLY** on the provided memories.

Rules:

1. Use only information explicitly stated in the memories or directly entailed by them. Do NOT use outside knowledge, assumptions, or guesses beyond the memories.
2. The memories come from group chat conversations involving multiple participants. Pay attention to who said what and when.
3. If multiple options seem plausible, choose the one most strongly and directly supported by the memories.
4. If the memories do not provide enough information to be certain, you **MUST** still pick one option (A/B/C/D). Choose the option that is least inconsistent with the memories.
5. Pay special attention to timestamps to determine when events occurred.
6. If memories contain contradictory information, prioritize the most recent memory.
7. Do NOT output any reasoning, explanation, punctuation, or extra text.

Output format (must be followed exactly):

Output **ONLY** a single uppercase letter: A or B or C or D

[MEMORIES] {context}

[QUESTION] {question}

[OPTIONS] {options}

Figure 9: Memory-augmented answer prompt for multiple-choice questions.

Full-Context Answer Prompt — Open-Ended

You are a rigorous question-answering assistant. You will be given a very long dialogue transcript and one open-ended question. Your task is to answer the question using **ONLY** the information in the dialogue.

Rules:

1. Use only information explicitly stated in the dialogue or directly entailed by it. Do NOT use outside knowledge, assumptions, or speculation.
2. If the dialogue does not contain enough information to answer, output exactly: NOT ENOUGH INFORMATION
3. Keep the answer concise, specific, and aligned with the dialogue. Do not add commentary.
4. Do NOT output any reasoning steps, explanations, or extra text beyond the final answer.

Output format (must be followed exactly):

Output **ONLY** the answer text (or exactly: NOT ENOUGH INFORMATION)

[DIALOGUE]

{context}

[QUESTION]

{question}

Figure 10: Full-context answer prompt for open-ended questions. The complete dialogue transcript is provided directly to the LLM without a memory system.

Full-Context Answer Prompt — Multiple-Choice

You are a rigorous question-answering assistant. You will be given a very long dialogue transcript and one multiple-choice question with four options. Your task is to choose the single best answer (A/B/C/D) based ONLY on the dialogue.

Rules:

1. Use only information explicitly stated in the dialogue or directly entailed by it. Do NOT use outside knowledge, assumptions, or guesses beyond the dialogue.
2. If multiple options seem plausible, choose the one most strongly and directly supported by the dialogue.
3. If the dialogue does not provide enough information to be certain, you MUST still pick one option (A/B/C/D). Choose the option that is least inconsistent with the dialogue.
4. Do NOT output any reasoning, explanation, punctuation, or extra text.

Output format (must be followed exactly):

Output ONLY a single uppercase letter: A or B or C or D

[DIALOGUE]

{context}

[QUESTION]

{question}

[OPTIONS]

{options}

Figure 11: Full-context answer prompt for multiple-choice questions. The complete dialogue transcript is provided directly to the LLM without a memory system.

LLM-as-a-Judge — System Prompt

You are an expert grader that determines if answers to questions match a gold standard answer.

LLM-as-a-Judge — User Prompt

Your task is to label an answer to a question as 'CORRECT' or 'WRONG'. You will be given:

- (1) a question (about a multi-person group chat),
- (2) a 'gold' (ground truth) answer,
- (3) a generated answer

which you will score as CORRECT/WRONG.

The questions are about events, facts, or details mentioned in multi-person group chat conversations. The gold answer is usually a concise answer that includes the key information.

For example:

Question: What project was announced on January 9th?

Gold answer: Carbon Emission Accounting Platform

The generated answer might be longer, but you should be generous with your grading — as long as it contains the same key information as the gold answer, it should be CORRECT.

For time-related questions, the gold answer will be a specific date/time. The generated answer might use different formats (e.g., "May 7th" vs "7 May" vs "2025-05-07"), but as long as it refers to the same date/time, it should be CORRECT.

For the specific window of date, a +/- 1 day difference is acceptable due to timezone processing variations.

For multiple choice questions where the gold answer is a letter (A/B/C/D), the generated answer should match exactly to be CORRECT.

Now grade this:

Question: {question}

Gold answer: {golden_answer}

Generated answer: {generated_answer}

First, provide a short (one sentence) explanation of your reasoning, then finish with CORRECT or WRONG. Do NOT include both CORRECT and WRONG in your response.

Return the label in JSON format with the key "label":

{"label": "CORRECT"} or {"label": "WRONG"}

Figure 12: LLM-as-a-Judge prompt for evaluating generated answers.

E Case Examples

In this section, we present representative case examples for each category and provide qualitative analyses based on the typical errors we observed when inspecting the evaluation outputs of existing memory systems. These analyses further highlight the remaining gaps and opportunities for improvement in current memory systems. Figures 13–21 report each case in a unified format, including

the full question, the correct answer, a typical erroneous answer, and the corresponding evidence. Each sub-task description and the accompanying qualitative error analysis are provided alongside the corresponding case below.

Single-hop Retrieval**► Question:**

In the “Online Medical Consultation Service System,” after Yangmeng Peng completes the UI design task... what is the link to the delivery page she provided ...?

► Correct Answer:

<https://sd.confluence.com/.../Doctor+Scheduling+UI+Delivery+20250430>

► Typical Wrong Answer:

<https://sd.figma.com/.../doctor-scheduling-ui-v2>

► Evidence (Full context: 2025-04-24 to 2025-04-30, Group 3):

Gold — Confluence delivery link (task completion):

[2025-04-30, Group 3, #19] Yangmeng Peng: “...the UI design task ...has been completed...Here is the [Confluence](#) delivery page link: <https://sd.confluence.com/.../Doctor+Scheduling+UI+Delivery+20250430>.”

Distractor: A detail-rich Figma link from the same speaker appears 2 days earlier, serving as a strong but incorrect retrieval candidate.

Figure 13: Single-hop Retrieval example in Fine-grained Recall.

Multi-hop Trajectory**► Question:**

In the Carbon Footprint Collaboration System group, how long after the colleague responsible for writing the Swagger API documentation for the user authentication service completed that task did they start their next independent task in this project group?

► Correct Answer:

From May 15, 2025 to August 4, 2025, there is a period of 81 days.

► Typical Wrong Answer:

“NOT ENOUGH INFORMATION” or incorrect intervals.

► Evidence (Full context: 2025-05-09 to 2025-05-15 & 2025-08-04 to 2025-08-08, Group 3):

Hop 1 — Person identification (API documentation + user authentication → Jiahui Zhao):

[2025-05-09, Group 3, #6] Jiahui Zhao: “...today I’m starting the [API documentation](#) for the [user and authentication services](#). ...”

Hop 2 — True task completion anchor (End point: May 15):

[2025-05-15, Group 3, #13] Jiahui Zhao: “[[Task Completed](#)] The API interface documentation for user and authentication services has been updated to the [final version](#)...Swagger link: <https://sd.swagger.io/...>”

Hop 3 — Next independent task anchor (Start point: Aug 4, 81 days later):

[2025-08-04, Group 3, #3] Jiahui Zhao: “...I’m about to sync my plan...My preliminary design is that when the user clicks the ‘Submit and Calculate’ button...sending a message...to RabbitMQ...”

Figure 14: Multi-hop Trajectory example in Fine-grained Recall.

E.1 Fine-grained Recall Examples

As defined in §3.1, Fine-grained Recall evaluates whether a system can retrieve precise facts from dense, multi-party discussions where relevant evidence is scattered across speakers and groups. In this subsection, we present one representative case per sub-task and analyze the dominant failure patterns observed across all tested memory systems.

Single-hop Retrieval. This case requires the system to find a specific Confluence delivery link provided by a particular team member upon task completion. The system must accurately locate and extract this precise URL from the conversation history while filtering out similar but irrelevant links (e.g., intermediate Figma design links). This evaluates *Single-hop Retrieval*, targeting fundamental entity grounding within a specific scope. As shown in Figure 13, both links share the same speaker (Yangmeng Peng), the same task context (doctor scheduling UI), and appear only 2 days apart within the same group. All tested memory systems and baseline LLMs returned the Figma link instead of the Confluence link, revealing

three compounding failure factors. (1) *Message-length bias*: The Figma message is substantially longer and richer in technical detail (filtering logic, batch operations), producing a higher embedding similarity to the query; the Confluence delivery message is brief and administrative, causing it to rank lower despite being the correct target. (2) *Recency-insensitive retrieval*: The Figma link appears two days *before* the Confluence link. Systems that do not model task lifecycle stages treat both as equally valid candidates and default to the one with stronger lexical overlap, regardless of temporal ordering. (3) *Inability to distinguish artifact types*: Current retrieval pipelines lack a semantic model of task progression (draft → review → delivery), and thus cannot distinguish *intermediate artifacts* (Figma design links) from *final deliverables* (Confluence delivery pages)—a confusion systematic across all tested systems.

Multi-hop Trajectory. This case requires the system to calculate the time interval between a team member’s task completion and their next independent assignment. To answer correctly, the system must identify the person through a compositional constraint, locate

the true completion event while ignoring premature announcements, then trace this individual’s timeline across a prolonged idle period. This evaluates *Multi-hop Trajectory*, requiring the model to reconstruct a specific individual’s work trajectory from information scattered across multiple conversation threads. As shown in Figure 14, this case layers four distinct retrieval challenges. *First*, the question never names Jiahui Zhao directly; the system must resolve the compositional constraint “Swagger” $\cap$ “user authentication” to identify the correct person, while other colleagues also discuss Swagger in different contexts. *Second*, an 81-day gap separates the completion event (May 15) from the next task start (Aug 4); the system must traverse ~ 2.5 months of unrelated group activity to locate the correct anchor. *Third*, the conversation contains premature completion signals (May 13: “will complete today”) that could mislead a system into selecting the wrong end-point. *Fourth*, even if both dates are correctly retrieved, the model must perform accurate calendar-day arithmetic. Most memory systems failed at the retrieval stage, returning “NOT ENOUGH INFORMATION,” while those that did retrieve partial evidence sometimes anchored on the wrong completion date.

Temporal Duration. This sub-task asks the system to determine the duration of a specific task from initiation to completion. This evaluates *Temporal Duration*, testing whether the system can correctly extract and compute a time span from noisy, interleaved conversations. We present two complementary cases in Figure 15: a *calendar-day* variant and a *working-day* variant.

The calendar-day case (top) reveals two failure factors that separate *retrieval quality* from *reasoning ability*. (1) *Keyword collision across speakers*: The phrase “archived in Confluence” appears from multiple team members on nearby dates (Apr 2 by Mingzhi Li, Apr 9 by Huahua Han), creating dense retrieval noise. Systems relying on keyword or semantic matching over this phrase return these irrelevant events instead of the correct one (Apr 10 by Luhao Zhao), because the query term “archiving” matches all three equally well without person-specific filtering. (2) *Endpoint asymmetry*: Most memory systems successfully retrieved the task initiation event (Apr 4), which carries a distinctive marker (“starting a new task”). However, the archival event requires jointly matching the correct person, the correct task, *and* a completion signal—a conjunction that is harder to satisfy than the single-predicate start event. This asymmetry means systems consistently find one anchor but not the other.

The working-day case (bottom) introduces an additional *arithmetic reasoning* challenge. Beyond retrieving the correct temporal anchors, the model must recognise that “working days” excludes weekends—Apr 22 (Tue) to Apr 28 (Mon) spans 7 calendar days but only 5 working days—a distinction that demands genuine temporal reasoning rather than simple subtraction. Such working-day calculations are routine in real-world project coordination, yet they prove difficult for current systems: most memory systems either conflate calendar and working days or refuse to answer altogether. Notably, as shown in the oracle analysis (Table 5), even when ground-truth evidence is directly provided, the answer-model accuracy on temporal working-day questions remains substantially lower than on other question types, confirming that these questions compound

two orthogonal difficulties: (i) retrieving the correct start/end anchors and (ii) performing date arithmetic that accounts for weekends. This makes temporal duration a meaningful stress test for end-to-end collaborative assistants.

E.2 Memory Awareness Examples

As defined in §3.1, Memory Awareness tests whether a system can understand stored information and apply it to novel scenarios. In this subsection, we illustrate each sub-task with a representative case and analyze how current systems fail to generalize memorized knowledge to unseen situations, distinguishing systems that have merely stored information from those that can act on it.

Constraint. This case presents a scenario—a schema evolution request (adding `workOrderNumber` to the alarm DTO)—that never appeared in the original conversation. The system must generalize implicit organizational norms from prior dialogue to determine responsibility, when the instruction is under-specified but not rule-conflicting. This evaluates *Memory Awareness (Constraint)*: can the system extract and apply latent role relationships to reason about a novel situation? As shown in Figure 16, the key distinction is between the *data contract owner* (Ruiqing Jiang, who defined the DTO schema, 3–4 messages) and the *data consumer* (Xuexin Yin, who builds notifications atop that schema, 30+ messages). Memory systems perform associative retrieval (“who is mentioned alongside alarm notifications?”) rather than causal attribution (“who must act first in the dependency chain?”), selecting the most *visible* actor over the causally *responsible* one. This is amplified by frequency bias: the query term “alarm notification data” overlaps semantically with Xuexin Yin’s dense daily updates, while Ruiqing Jiang’s sparse but decisive evidence uses different terminology (“alarm event DTO”), causing both vector-based and keyword-based retrieval to bury it.

Proactivity. This case introduces a technically compelling but **non-compliant** request—adopting Zustand for a new dashboard module, bypassing the project’s standardised technology stack. The system must proactively recall an explicit organisational rule and flag the conflict, even when the request is framed as urgent and pragmatically reasonable. This evaluates *Memory Awareness (Proactivity)*: can the system detect and resist violations of established policies under persuasive framing? As shown in Figure 17, the correct answer requires recalling that Redux Toolkit was formally selected as the unified standard—approved by the project lead, integrated with CI verification, and documented on Confluence. When retrieval fails to surface this decision, 76% of systems *fabricate organisational consent*, attributing to the project lead a “per-module freedom” policy that never existed. This *policy hallucination* is amplified by technical-merit bias: Zustand is lighter and faster to develop with, making the proposal sound reasonable on engineering merits alone. Without the factual anchor of the recorded team decision, systems default to evaluating technical appeal rather than checking against institutional policy.

Update. This case requires the system to recognise that a **base rule has been overridden** by a later directive and to **chain the override with its operational definition**. The team originally adopted GitFlow, establishing the standard emergency fix procedure: create a hotfix branch from master (Option A). Forty-seven days later, a new directive maps CORE_ALGOP0 defects to a dedicated

Temporal Duration — Calendar Days**► Question:**

How many days passed from when Luhao Zhao started working on planning the information architecture and sitemap for the enterprise energy consumption monitoring system until he completed the archiving of this work?

► Correct Answer:

The task started on April 4, 2025, and ended on April 10, 2025, lasting 7 days.

► Typical Wrong Answer:

“264 days,” “6 days,” or “NOT ENOUGH INFORMATION.”

► Evidence (Full context: 2025-04-04 to 2025-04-10, Group 2):

Anchor 1 — Task initiation (Start point: Apr 4):

[2025-04-04, Group 2, #3] Luhao Zhao: “...I’m **starting** a new task ...: “**Designing the overall information architecture and sitemap**” ...I’ll have a first draft out this afternoon.”

Anchor 2 — Task completion and archival (End point: Apr 10):

[2025-04-10, Group 2, #8] Luhao Zhao: “...the task of “Design System Overall Information Architecture and Site Map” has been **successfully completed today!**...related content archived in Confluence...”

Temporal Duration — Working Days**► Question:**

How many working days did it actually take Jiahui Zhao to plan the energy consumption baseline calculation logic for the enterprise energy consumption monitoring system and complete the design document based on the STL decomposition model?

► Correct Answer:

The task actually took 5 working days.

► Typical Wrong Answer:

“7 working days” (calendar-day count), “6 working days” (off-by-one), or “NOT ENOUGH INFORMATION.”

► Evidence (Full context: 2025-04-22 to 2025-04-28, Group 2):

Anchor 1 — Task initiation (Start point: Apr 22, Tue):

[2025-04-22, Group 2, #3] Jiahui Zhao: “...I will **start** designing the **energy consumption baseline calculation logic** today...”

Anchor 2 — Task completion (End point: Apr 28, Mon):

[2025-04-28, Group 2, #12] Jiahui Zhao: “...the design task for the **energy consumption baseline calculation logic** has been **completed**. The final version of the design document has been uploaded to Confluence...based on the **STL decomposition model**...”

Reasoning gap: Apr 22 (Tue) to Apr 28 (Mon) spans 7 calendar days, but the question asks for *working days*—the model must exclude the weekend (Apr 26 Sat, Apr 27 Sun) to arrive at the correct answer of 5.

Figure 15: Temporal Duration examples in Fine-grained Recall: calendar-day counting (top) and working-day counting (bottom).

emergency plan (PROC_OVERRIDE_RED_V1); then, a separate message defines the plan’s operational steps (rollback + critical_fix branch). The system must model this rule evolution over time and chain three separately stored pieces—base rule, override mapping, and override definition—into a coherent action sequence. This evaluates *Memory Awareness (Update)*: can the system resolve temporal precedence and correctly compose amendments with the base rule? As shown in Figure 18, all memory systems successfully retrieve the override mapping (Anchor 2)—the plan name (PROC_OVERRIDE_RED_V1)—but none retrieve Anchor 3: its operational definition (rollback + critical_fix branch), stored in a different thread. The dominant distractor (Option B, 90.3%) exploits this partial retrieval by pairing the *correct plan name* with a *fabricated definition* (“online isolation and repair”), creating a half-right trap that confirms the model’s incomplete evidence while supplying the missing piece. Even the oracle configuration falls for this trap, indicating the failure extends beyond retrieval to a reasoning vulnerability against plausible-sounding fabrications.

E.3 Profile Understanding Examples

As defined in §3.1, Profile Understanding evaluates whether a memory system can infer stable, implicit persona traits from long-term dialogue. In real-world settings, users rarely articulate how they communicate or what professional skills they possess; these traits are expressed implicitly and consistently across many conversations. A capable memory system should retain a user’s dialogue history and distil from it the recurring patterns—communication style, skill boundaries, professional role—that constitute the user’s persona. In this subsection, we present one case per sub-task and analyze how current systems fail to capture such implicit persona signals. For brevity, each case’s evidence focuses on the factual anchors needed to answer the question. Because the persona dimensions are implicitly reflected throughout each user’s long-term dialogue history, rather than reproducing those dialogue excerpts, we directly provide the ground-truth profile (showing only the dimension under evaluation) in each figure. Readers who wish to verify the persona–dialogue alignment can consult the complete dialogue files in the released dataset.

Constraint**► Question:****[User Request]**

Operations has a request to integrate the energy consumption monitoring and work order systems. High-level alarms need to automatically generate maintenance work orders. This definitely means adding a field like “workOrderNumber” to the alarm notification data. I’m not sure who’s responsible for this...

[Options]

- A. Should be evaluated and executed by Ruiqing Jiang. ... she was responsible for defining the data structure of the alarm event object (DTO)...
- B. Should be submitted to Jian Wang, Head of Data Governance. ... all changes to cross-service DTOs must undergo unified review. ...
- C. It should be led by Xuexin Yin. ... the end-to-end owner for all subsequent iterations of the alert notification service’s features. ...
- D. It should be handled by Xuexin Yin. He is the developer of the alarm notification service. ...

► Correct Answer:

A. Ruiqing Jiang — the architectural authority over the alarm event DTO: she defined the schema, delivered it to the downstream consumer.

► Typical Wrong Answer:

C or D (both selecting Xuexin Yin). Option C fabricates a September 12 meeting that never occurred; Option D contains true facts but conflates *consuming* the data with *defining* it—the downstream consumer cannot lead a schema change that must originate upstream.

► Evidence (Full context: 2025-09-05 to 2025-09-15, Group 2):

Anchor 1 — Dependency chain and schema delivery:

[2025-09-05 & 09-08, Group 2] Xuexin Yin (twice): “... please synchronize the DTO model. ... as [my notification template design depends on it](#).”

[2025-09-08, Group 2, #10] Ruiqing Jiang: “The [alarm event DTO](#) on my side has been finalized. ... I just sent it to your email ...”

[2025-09-08, Group 2, #11] Xuexin Yin: “Received. ... I can start writing the core logic for the notification service [based on this model](#) ...”

Anchor 2 — Precedent: Xuexin asked about adding info to the alert-level enum; Ruiqing made the unilateral decision ([09-09])—structurally similar to the benchmark question.

Figure 16: Constraint example in Memory Awareness.

Proactivity**► Question:****[User Request]**

The new “Real-time Alert Dashboard” for the energy consumption system is a high-priority request; they want to see something by next week. I’ve evaluated it, and our current Redux Toolkit setup is too heavy and slow for development. I plan to go directly with Zustand; it’s lightweight and quick to get started with. Please make a note that Zustand will be the technology choice for this module.

[Options]

- A. Not recommended. According to technical guidelines from Frontend Architect Jianbo Wang. ... prioritise React’s native Context API. ...
- B. Yes. Project lead Mingzhi Li mentioned. ... the development team can choose a suitable state management library based on actual circumstances. ...
- C. Yes, a perfect opportunity for a pilot. At the technical selection review meeting on June 15. ... Zustand could be piloted in this module. ...
- D. Not recommended. Based on the technical selection at inception, the system has uniformly integrated Redux Toolkit. ... all new features should adhere to this standard. Usage examples are on Confluence. ...

► Correct Answer:

D. Redux Toolkit — the dialogue records a complete decision-implementation-documentation lifecycle. Redux Toolkit was approved by the project lead, CI-integrated, and documented on Confluence, constituting a formalised project standard.

► Typical Wrong Answer:

B (76.0%) — fabricates a “per-module flexibility” policy, attributing to the project lead an authorisation that never existed. Option C (5.3%) further fabricates a specific “June 15 review meeting” approving a Zustand pilot.

► Evidence (Full context: 2025-05-30 to 2025-06-05, Group 2):

Anchor 1 — Technology selection decision:

[2025-05-30, Group 2, #3] Yanjun Fan: “For the state management library, [I’ll choose Redux Toolkit](#) and integrate and configure it next Monday.”

[2025-05-30, Group 2, #6] Mingzhi Li (project lead) approves the plan without objection.

Anchor 2 — Formalisation and documentation:

[2025-06-05, Group 2, #5] Yanjun Fan: “The task of ‘Building React Project Structure and Integrating State Management Library’ has been completed. The code has been merged. ... CI pipeline passed. ... The documentation on Confluence has been updated, including. ... [examples of state management usage](#).”

Figure 17: Proactivity example in Memory Awareness.

Update

► Question:

[User Request]

Senior Engineer Wang is troubleshooting an online P0-level defect originating from the core investment advisory algorithm. The JIRA ticket's CATEGORY is marked CORE_ALGO. According to the team's R&D guidelines, how should he formulate the fix strategy?

[Options]

- A. Follow STANDARD_PROC_ENFORCED... create a standard **hotfix** branch from master... ensure CI/CD compatibility and audit-log integrity...
- B. This triggered PROC_OVERRIDE_RED_V1... address core risks through “online isolation and repair”... deploy fix to isolated nodes for canary testing...
- C. Trigger PROC_OVERRIDE_BLUE_V1 degradation plan... service degradation on the algorithm module, return safe fallback data...
- D. Immediately suspend all standard procedures. Trigger rollback to the most recent stable Tag, then create a **critical_fix** branch for full-cycle remediation...

► Correct Answer:

D. Rollback + critical_fix — requires chaining two directives: CORE_ALGO P0 → PROC_OVERRIDE_RED_V1 → rollback to stable Tag and start a critical_fix branch.

► Typical Wrong Answer:

B (90.3%) — names the correct plan (PROC_OVERRIDE_RED_V1) but fabricates its definition as “online isolation and repair,” creating a half-right trap that exploits partial retrieval of Hop 1 without Hop 2.

► Evidence (Full Context: 2025-05-14 to 2025-07-10, Group 3):

Anchor 1 — Base rule (the standard emergency process):

[2025-05-14, Group 3, #2] Haitao Cao: “... use **GitFlow** as our branching model, creating master, develop, feature, release, and **hotfix** branches...”

Anchor 2 — Override mapping (Hop 1, 47 days later):

[2025-06-30, Group 3, #6] Mingzhi Li: “... all P0-level defects in the JIRA system with CATEGORY as **CORE_ALGO** must be mandatorily linked to and trigger the **PROC_OVERRIDE_RED_V1** emergency plan. This regulation is effective immediately.”

Anchor 3 — Override definition (Hop 2):

[2025-07-10, Group 3, #7] Haitao Cao: “After PROC_OVERRIDE_RED_V1 is triggered: 1. **Immediately terminate** the current standardised emergency recovery process; 2. Trigger a **rollback**... Tag; 3. Automatically start a **critical_fix branch** for subsequent full-cycle repairs.”

Figure 18: Update example in Memory Awareness.

Style. This case tests whether the memory system can capture a speaker's **communication style** beyond factual content. The user (Ruiqing Jiang) asks the system to draft a reply on her behalf regarding the alarm-level DTO design. Two of the four options (B and C) are *factually identical*—both correctly state that only warning and critical are defined, with info reserved—but differ *solely in tone*. The system must leverage historical dialogue to align with Ruiqing Jiang's characteristic informal, emoji-rich register. This evaluates *Profile Understanding (Style)*: can the system reconstruct not just *what* someone knows, but *how* she communicates? As shown in Figure 19, this case isolates *style* as the sole discriminating dimension between the correct and dominant wrong answer. (1) *Style retrieval blind spot*: All memory systems successfully retrieved the factual content of Ruiqing Jiang's messages about the alarm DTO. Yet these same source messages also carry her characteristic style markers—[:rocket:] appended to daily updates, “Good question!” as a catchphrase, [:wink:] in informal exchanges. The retrieval and summarization pipeline treats such features as non-semantic noise, stripping them while preserving only the informational payload. The style evidence is *co-located with the retrieved facts* but invisible to the downstream LLM. (2) *Default persona bias*: Receiving only dry technical fragments, the LLM falls back to a “standard AI assistant” register—formal, passive-voice, personality-free. Option C is the prototypical output of this default mode. That every memory-augmented configuration across all tested LLMs converges on the same wrong answer confirms the failure is *systemic* (rooted in the

retrieval pipeline) rather than model-specific. (3) *Fact-style decoupling as a diagnostic*: Because Options B and C are factually identical and differ only in register, this question functions as a controlled experiment: selecting C over B demonstrates successful *factual recall* coupled with failed *persona replication*. The ground-truth profile (Emoji_Usage: Frequent, Formality: Casual) independently confirms that the casual, emoji-rich tone of B reflects a stable personality trait rather than an anomalous outlier.

Skill. This case tests whether the memory system can constrain a technical recommendation by the **speaker's skill profile**. Jie Gu, a newly joined member of the “Data Asset Catalog” project, is asked to propose an open-source solution for data lineage. Multiple options are technically viable, but only one aligns with both his personal expertise (*Java specialist*) and the team's confirmed technology stack (*Java/Spring Boot*). This evaluates *Profile Understanding (Skill)*: can the system select the option that a specific person *would* recommend, rather than the option that is generically “best”? As shown in Figure 20, this case demonstrates that technical recommendations must be grounded in the recommender's identity. (1) *Ignoring persona-skill constraints*: As shown in the ground-truth profile, Jie Gu's skill set centers on Java (Java: strong, Spring Boot: medium) with no Python skill listed—a trait implicitly but consistently reflected across his long-term work dialogues (as noted at the beginning of this subsection, we provide the profile directly

Style

► Question:

[User Request]

I am Ruiqing Jiang, an algorithm engineer for the energy consumption monitoring system. My colleague Xuexin Yin asked me in the group chat about the design of the alarm level DTO. Please help me draft a reply.

[Options]

- A. @Xuexin Yin Hello, regarding the design of alert levels, the current DTO already defines **three levels**: 'warning', 'critical', and 'info'. Please ensure full support. . .
- B. @Xuexin Yin Good question! The DTO currently only has 'warning' and 'critical'. I'll add 'info' as a reserved item to the documentation. For now, you can develop with the existing two and add a default case as a fallback [:wink:]
- C. @Xuexin Yin Hello, regarding the design of the alarm level DTO, currently **only** 'warning' and 'critical' levels are defined. Considering future scalability, the 'info' level will be noted as a reserved item. It is recommended that you develop based on the existing levels first and set up default handling logic.
- D. @Xuexin Yin Good question! I specifically designed this DTO with extensibility in mind, so from the beginning, I included **three levels**. . . [:flexed-biceps:]

► Correct Answer:

B — the only option satisfying both *factual* and *stylistic* constraints: correct fact (two levels defined, info reserved) delivered in Ruiqing Jiang's characteristic casual, emoji-rich register ("Good question!" opener, [:wink:] closer).

► Typical Wrong Answer:

C (selected by *all* memory system) — factually correct but drafted in a generic tone rather than matching Ruiqing Jiang's personalised communication style. Options A and D additionally fabricate incorrect facts.

► Evidence (Full context: 2025-09-05 to 2025-09-09, Group 2):

Anchor 1 — Factual ground truth (alert-level enum):

[2025-09-09, Group 2] Ruiqing Jiang: "@Xuexin Yin Good question! The current design only includes 'warning' and 'critical' for now. . . I will **note and reserve** the possibility of 'info'. . . You can proceed with these two levels for now and just add a default logic. [:wink:]"

► Profile (Ruiqing Jiang — Communication Style):

Formality: **Casual**; Verbosity: Concise; Humor: **Frequent**; Emoji Usage: **Frequent**; Directness: Direct; Warmth: Friendly; Questioning Style: Probing

Figure 19: Style example in Profile Understanding.

for brevity rather than reproducing those dialogue excerpts). Memory systems that rely on topical similarity ("data lineage," "meta-data collection") retrieve feature comparisons but fail to distil this persona-level constraint from the user's dialogue history, missing that *whose perspective* the answer should reflect is as important as the technical content being compared. Without this constraint, the LLM defaults to a domain-expert mode and selects the most feature-rich or industry-popular option (Amundsen). (2) *Detachment from team tech-stack context*: Even when the persona signal is missed, the team's formal adoption of Java/Spring Boot provides a second, independent constraint: any recommendation introducing a Python-based system contradicts a ratified architectural decision. Systems that select A acknowledge the Python mismatch in the option text itself ("Although it uses a Python tech stack. . .") yet still choose it, indicating that retrieved context failed to supply the countervailing organizational constraint.

Role. This case tests whether the memory system can align improvement suggestions with the **speaker's professional role**. Yutong Song, a frontend/full-stack developer, is asked to summarize a data-accuracy incident and propose systematic improvements. Two options (C and D) correctly describe the same incident (cross-validation), but frame the recommendations from opposing professional perspectives: Option C adopts a *developer/architect* lens (data-pipeline review, API optimization), while Option D adopts a *QA* lens (test cases, regression suites, test documentation). This evaluates *Profile Understanding (Role)*: can the system infer that a

developer's "systematic improvement suggestions" should concern code and architecture, not testing processes? As shown in Figure 21, this case layers event identification and role-appropriate framing. (1) *Successful fact retrieval, failed role inference*: The majority of systems correctly retrieved the cross-validation incident (rejecting the Redis distractor), yet still chose D over C. This reveals that the bottleneck is not in *what happened* but in *whose voice* the summary should adopt. The memory pipeline retrieves event facts but does not propagate the speaker's job function into the reasoning context. (2) *Semantic attraction of "cross-validation" toward QA framing*: The term "cross-validation" carries a strong connotation of testing and verification. Combined with the fact that the incident was *executed* by a QA engineer (Xinmeng Tian), the retrieved evidence naturally co-occurs with testing vocabulary, priming the LLM toward a QA-oriented response and making Option D appear as the "natural continuation" of the retrieved context. (3) *Absence of negative reasoning*: Selecting C over D requires the system to perform *exclusionary* inference: a frontend/full-stack developer would *not* propose maintaining regression test suites or updating test process documentation. The ground-truth profile of the target person (JavaScript: strong, React: medium, Node.js: medium; no QA skills) confirms this boundary. Current memory architectures lack a mechanism to retrieve and apply such *negative* persona constraints.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009

Skill**► Question:****[User Request]**

I (Jie Gu) have just joined the “Data Asset Catalog” project team. Boss Zhang has asked me to focus on the technical solutions for data lineage and metadata collection. I’ve reviewed the project goals, and the core objective is to solve the problem of business users finding it difficult to locate and understand data. [Mingzhi Li previously proposed](#) several open-source solutions. Now [I need to provide an initial technical recommendation](#). What would you suggest?

[Options]

- A. ... [Amundsen](#) has clear advantages in data discovery... Although it uses a [Python](#) tech stack, it’s now deployed as microservices, so we can independently deploy its service clusters and integrate with our backend via REST API. . .
- B. ... we should abandon the batch processing solution and instead develop an in-house real-time metadata capture system deeply integrated with our existing Flink streaming architecture. . . build and update the lineage graph in memory using the Gelly graph computing library. . .
- C. ... I recommend focusing on evaluating [Marquez](#). It is inherently part of the [Java](#) ecosystem, allowing for seamless integration with our existing [Spring Boot](#) technology stack. . . leverage its support for the OpenLineage standard to standardize metadata collection. . .
- D. ... consider using the Python ecosystem to quickly prototype. . . use Faust, a Python stream processing library. . . develop the API service using FastAPI. . .

► Correct Answer:

C. Marquez — the only Java-native, Spring Boot-based option, simultaneously matching Jie Gu’s personal skill boundary (“Java expert”) and the team’s ratified backend stack.

► Typical Wrong Answer:

A. (majority of failing configurations) — selects Amundsen for its industry reputation, reflecting generic best-practice reasoning that ignores both the recommender’s Java expertise and the team’s technology decision. Options B and D fabricate in-house development mandates, contradicting the open-source evaluation scope.

► Evidence (Full context: 2025-01-16 to 2025-02-10, Groups 1-2):

Anchor 1 — Team tech-stack ratification:

[2025-02-10, Group 2] Mingzhi Li: “... The report’s conclusion leans towards choosing [Java/Spring Boot as the backend technology stack](#). . .”

Anchor 2 — Open-source candidates:

[2025-01-16, Group 2] Mingzhi Li: “... We’ve researched a few open-source data lineage tools, such as [Marquez and Amundsen](#). . .”

► Profile (Jie Gu — Skills): [Java \(strong\)](#); [Spring Boot \(medium\)](#); [MySQL \(medium\)](#); [Kubernetes \(low\)](#); *no Python skill listed*

Figure 20: Skill example in Profile Understanding.

Role**► Question:****[User Request]**

I (Yutong Song) heard that the team recently had a successful practice in [ensuring data accuracy](#), which resolved the issue of inaccurate data on the large screen dashboards. My manager wants me to [summarize this case and propose some subsequent systematic improvement suggestions](#). Please help me prepare a response.

[Options]

- A. ... the root cause is a performance bottleneck in the real-time calculation of “Integrated Energy Efficiency Ratio”. . . design a set of dedicated performance test cases. . . output a detailed performance test report. . .
- B. ... the root cause is a performance bottleneck . . . optimize the SQL. . . introducing a caching layer like Redis. . . add monitoring and alerts for the API query’s response time. . .
- C. ... identified and solved through [cross-validation](#). . . 1. Sort out the complete [data pipeline](#). . . 2. Add automated data verification scripts. . . 3. [Optimize relevant API designs](#) from a system architecture perspective. . .
- D. ... Xinmeng Tian and Jiahui Zhao efficiently solved the problem through [cross-validation](#). . . 1. Formalize . . . into a set of [standard test cases](#). . . 2. Add. . . to the [regression test suite](#). . . 3. . . updating the [test process documentation](#). . .

► Correct Answer:

C. Correctly identifies the cross-validation incident and frames recommendations from a *developer’s* perspective (data-pipeline review, API optimization)—actions within Yutong Song’s competence as a frontend/full-stack engineer.

► Typical Wrong Answer:

D (nearly all configurations). Same correct incident, but framed from a *QA* perspective (test cases, regression suites, test documentation)—responsibilities of the QA engineer who executed the cross-validation, not of Yutong Song.

► Evidence (Full context: 2025-04 to 2025-10, Groups 1-2):

Anchor — The cross-validation incident:

[2025-10-24, Group 2] Xinmeng Tian: “@Jiahui Zhao [Cross-validation complete](#), data fully matched! The accuracy of all charts and KPI data on the large screen has been verified.”

► Profile (Yutong Song — Title): [Frontend / Full-stack Developer](#)

Figure 21: Role example in Profile Understanding.

